Project-Based Case Study

Design and Evaluation of the Technology Template “Template-Projekte”

Technical Investigation of a Reusable Foundation for Web, Cloud, and Desktop Projects

Project:	Template-Projekte
Author:	Marcel Tenhaft
Document version:	13 August 2026

Independently authored technical documentation and evaluation
of the “Template-Projekte” project

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AWS	Amazon Web Services
CI	Continuous Integration
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DB	Database
DEB	Debian package format
E2E	End-to-End
GCP	Google Cloud Platform
HPC	High Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PG	PostgreSQL
PID	Process Identifier
RBAC	Role-Based Access Control
SDK	Software Development Kit
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
ZIP	Compressed archive format

1 Introduction

1.1 Background

New software projects repeatedly face the same technical decisions. These concern architecture, technologies, interfaces, configuration, testing, build, and deployment. If these foundations are created anew each time, projects begin from differing technical baselines. At the same time, web, cloud, and desktop applications require both shared structures and platform-specific extensions.

The “Template-Projekte” project consolidates these foundations in a reusable full-stack technology template. It is intended to provide a common basis for web, cloud, and desktop projects (kleiveist, 2026l). Such a basis can consolidate recurring decisions and establish a consistent development model. To achieve this, shared components must be clearly separated from platform-specific extensions.

In simple terms: Types of software projects

A software project involves the planned development or modification of a computer program. A web application is generally opened in a browser. A cloud application uses services that run on servers and are accessed over a network. A desktop application, by contrast, runs on a computer and can interact directly with its operating system. The template under examination is intended to provide a common technical starting point for these different project types.

1.2 Problem Statement

Repeatedly initializing similar projects consumes effort outside product development. Without a shared baseline, structure, tools, configuration, and build processes differ; separate starting points increase maintenance effort and may diverge technically.

The template must therefore combine standardization with flexibility: a common basis should enable reuse without burdening projects with unnecessary components. This case study examines how project profiles and optional extensions address this trade-off and where adaptation is still required (kleiveist, 2026f, 2026g).

In simple terms: Templates and baselines

A template is a reusable blueprint from which new projects can be derived. The baseline is the shared technical starting state of this blueprint, including its structure, tools, and workflows. This means that recurring foundations do not have to be recreated for every project. Individual projects can then extend the common basis to suit their purpose.

1.3 Objectives

The case study examines the architecture, shared and profile-specific components, and the development and quality-assurance workflows of the “Template-Projekte” project (kleiveist, 2026f, 2026m).

Based on existing project artifacts and evidence, it assesses reusability, standardization, productivity potential, and suitability for different project types. From this assessment, it derives criteria for use

and selection, technical limitations, and external prerequisites (kleiveist, 2026k). The study does not investigate universal suitability for every class of software or a product-specific domain architecture.

In simple terms: Technical case study

A technical case study systematically examines a specific project in its existing state. Here, this includes examining its files, workflows, and technical evidence. This makes it possible to distinguish substantiated strengths from limitations and unresolved issues. The findings apply to the state under examination and do not prove that the template is suitable for every type of software.

1.4 Research Questions

Six questions structure the investigation and assessment:

1. Which functional and technical, quality, and operational requirements does the technology template address?
2. How do the architecture and profile model support the provision of different variants for web, cloud, and desktop projects?
3. To what extent do the shared technical core and the single-repository strategy promote reuse and standardization across projects?
4. What potential does the template offer for reducing recurring effort in project initialization, development workflows, quality assurance, and deployment?
5. For which project types and customer requirements does the template provide a suitable technical starting point?
6. Which technical limitations, maintenance efforts, risks, and external dependencies must be considered when using it?

Their answers are based on requirements, architecture, and the available verification evidence.

1.5 Methodology

The investigation follows a technical case-study approach (Runeson & Höst, 2009). An inventory records the structure, components, and boundaries of responsibility within the master repository. The subsequent architecture analysis examines the frontend, backend, desktop integration, interfaces, persistence, and deployment. It distinguishes substantiated properties from assumptions and unresolved verification points (kleiveist, 2026f).

The profile analysis examines the shared core, variants, and optional capabilities. In addition, the Python tooling is examined with respect to generation, system checks, installation, development, testing, builds, and release (kleiveist, 2026g, 2026m). Internal project descriptions of the architecture, profiles, and tools serve as primary sources for the documented state.

Existing tests, build results, and documented acceptance results are mapped to the requirements and architecture decisions. External or product-specific checks remain separate. This evidence underpins

the assessment and the transfer of findings to potential customer projects (kleiveist, 2026k). Figure 1 shows the sequence of the investigation. Assessment and transfer therefore take place only after the inventory, analysis, and verification; unresolved verification points are not treated as confirmed properties.

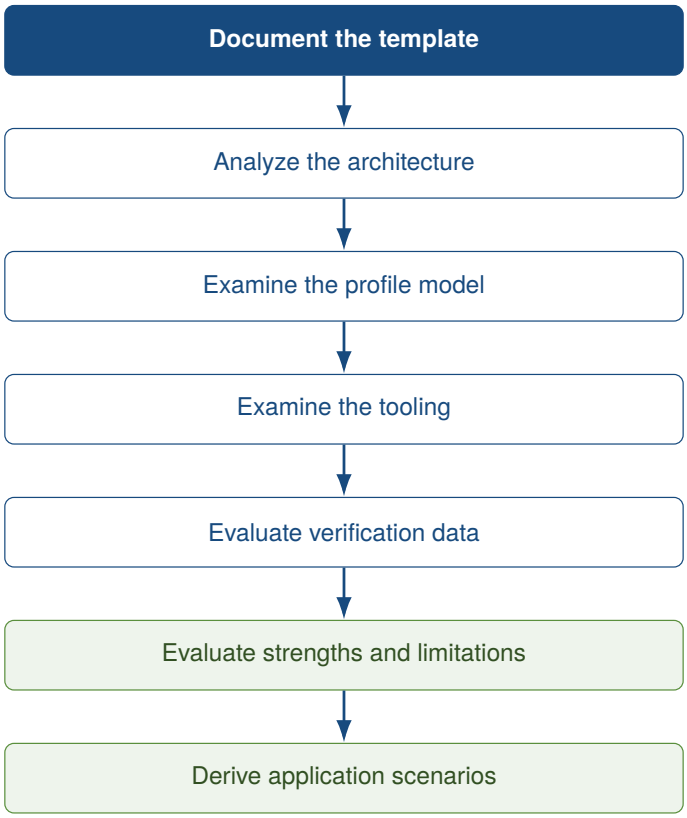

Figure 1: Methodological steps of the case study

Source: Author's own illustration.

In simple terms: How the investigation is conducted

Software architecture describes how a program is structured and how its parts interact. An analysis examines these parts and their relationships in a systematic manner. Verification means checking a technical claim through an appropriate check. Technical evidence is the traceable proof that supports the claim, such as a test or build result. In this case study, unverified points are therefore not treated as confirmed properties.

1.6 Structure of the Case Study

The requirements are followed by the architecture, project profiles, and tooling. The case study then addresses quality assurance and verification, assesses the observed state, and transfers the findings to customer projects. The conclusion answers the research questions and identifies limitations and prospects for further development.

2 Requirements for the Technology Template

2.1 Target Vision for the Technology Template

The template is intended to provide a reusable basis for web, cloud, and desktop projects. To this end, it combines planned commonality with controlled variability (Clements & Northrop, 2001; Krueger, 1992). A shared core is intended to centralize structure, workflows, tooling, documentation, configuration, tests, and interfaces.

Profiles select suitable combinations from this core; optional capabilities such as PostgreSQL extend compatible backend profiles (kleiveist, 2026g). The frontend, backend, desktop layer, persistence, tooling, and deployment remain separate areas of responsibility. At this stage, these statements describe requirements, not verification results.

In simple terms: Areas of technical responsibility

The frontend is the visible part of an application with which people interact. The backend operates in the background, processing requests or providing data. A desktop layer connects the application to an operating system and enables additional functions there. Persistence ensures that data remains available after a program is closed. Deployment means providing and starting the application in its target environment. The template is intended to separate these areas clearly; at this point, these are requirements rather than confirmed verification results.

2.2 Requirements for Reusability and Standardization

The baseline is intended to define shared structures and public workflows while allowing controlled profile variants. Centralized maintenance prevents copies from diverging. Table 1 summarizes the verifiable requirements.

Table 1: Key requirements for the technology template

ID	Requirement	Operationalized objective	Basis for subsequent verification
A-01	Baseline	It must be possible to generate the basic structure.	Project generation
A-02	Structure	Responsibilities are assigned to consistent locations.	Repository structure
A-03	Profiles	Only designated components are activated.	Profile model
A-04	Workflows	Checks, initialization, installation, startup, testing, and building share common entry points.	Tooling
A-05	Maintainability	Shared components are maintained centrally.	Repository strategy
A-06	Traceability	Changes and configuration are versioned and documented.	Version state, documentation
A-07	Extensibility	Optional capabilities have documented dependencies.	Capability model

Source: Author's own compilation.

The stated verification basis is subsequently used to assess these target criteria.

In simple terms: Shared instead of recreated each time

Reuse means applying existing structures or workflows in multiple projects. Standardization gives these projects shared rules and entry points so that they are structured and operated in similar ways. A central baseline consolidates these shared elements in one place where they can be maintained. This is intended to prevent separate templates from diverging technically without being noticed.

2.3 Requirements for Web, Cloud, and Desktop Applications

All variants are intended to share the frontend foundation, structure, configuration logic, tests, and documentation. Browser projects do not require a desktop layer and need a backend only when necessary. Cloud variants require an API, deployment boundaries, and health and readiness checks. Desktop variants use the same frontend in a native runtime; combined variants add a backend and, where applicable, browser access.

Table 2: Requirements matrix for the supported project types

Requirement	Web	Cloud	Local desktop	Desktop + Cloud
Frontend	required	required	required	required
Backend / API	optional	required	not inherently required	required
Desktop runtime	no	no	required	required
Deployment boundary	optional	required	no	required
Persistence	optional	optional	product-specific	optional

Source: Author's own compilation based on kleiveist (2026g).

PostgreSQL is an optional backend capability, not a platform profile; profiles determine neither the format, persistence provider, nor source of truth (kleiveist, 2026g, 2026h). Client and server configuration remain separate; no cloud platform is assumed.

In simple terms: Platforms and API

Here, web, cloud, and desktop refer to the browser-based, server-supported, and locally installed forms of application introduced above. Full stack describes a solution that comprises both the visible frontend and a backend. An API is a defined interface through which programs exchange requests and data. The template's profiles combine these areas in different ways; consequently, not every variant requires a backend or a database.

2.4 Quality and Maintainability Requirements

The requirements are guided by verifiable characteristics of ISO/IEC 25010 (ISO/IEC, 2023). Components and profiles should have clear responsibilities, be testable in isolation, and remain maintainable through documented decisions. Extensions and technology updates should not require a copy of the entire baseline, although they still entail risks.

Identical inputs and configurations should produce traceable generation and build results. This strengthens the verifiable relationship between the source-code state and the artifact (Lamb & Zacchiroli, 2022). It should be possible to generate target artifacts in a controlled manner, and configuration priorities should be comprehensible; server-side secrets remain separate from client configuration.

In simple terms: Quality objectives

Testability means that a component can be checked through clearly defined tests, as independently of other parts as possible. Maintainability describes how readily software can be understood, modified, and updated. Reproducibility means that the same defined inputs and settings produce traceable results. In the template, these are quality objectives that guide the subsequent technical verification.

2.5 Scope

The template is not a universal software platform. Its target scope encompasses recurring web, backend, and desktop projects, including full-stack business applications.

Business rules, domain and customer data models, and product-specific authentication, role, and authorization concepts are not part of the baseline. It also prescribes no product or user-data format, persistence provider, or cloud provider (kleiveist, 2026f, 2026h).

Highly platform-specific native applications, games, and highly specialized real-time systems fall outside the immediate target scope.

In simple terms: A baseline rather than a finished product

The scope defines what an investigation or solution includes and what lies outside its target area. Product-specific functions are those required only for a particular use case or customer. The baseline provides shared technical foundations but is not yet a finished product. Domain rules, concrete data models, and appropriate access rights must therefore be added to each product project.

3 Architecture of the Technology Template

3.1 Overall Architecture

The baseline consists of a master repository with a shared core and declarative variants. Profiles select reusable features; optional capabilities extend only compatible profiles. This controlled variability avoids separate copies of the template (Clements & Northrop, 2001; kleiveist, 2026g).

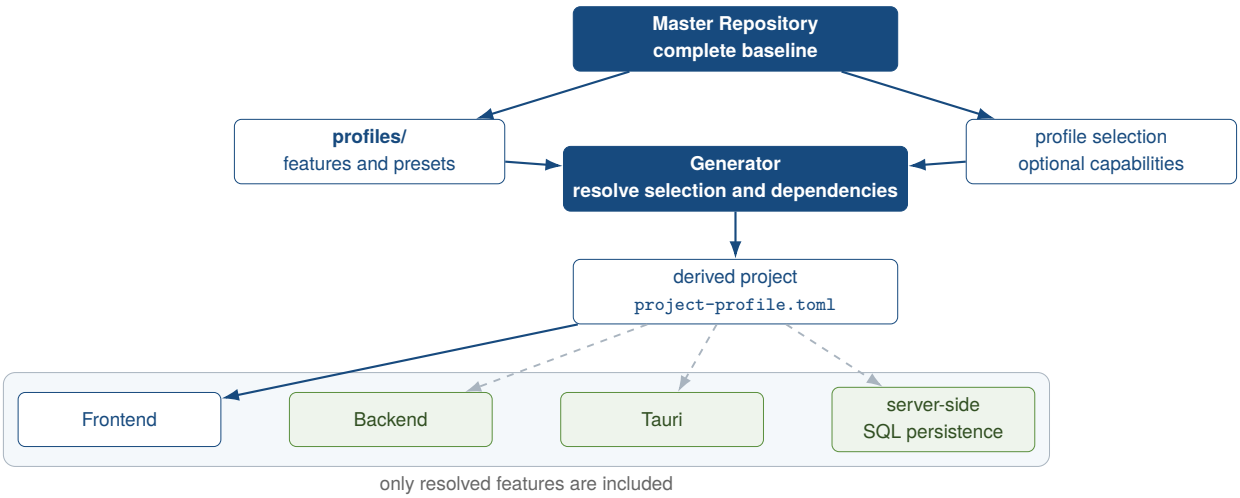

Figure 2: Overall architecture of the technology template

Source: Author's own illustration based on kleiveist (2026f, 2026g, 2026h).

Figure 2 distinguishes the baseline from the derived project. The generator resolves the profile and capabilities, copies the associated paths, and records the result in `project-profile.toml`. The frontend, backend, Tauri, and server-side SQL persistence remain independent components, some of which are optional (kleiveist, 2026f, 2026h).

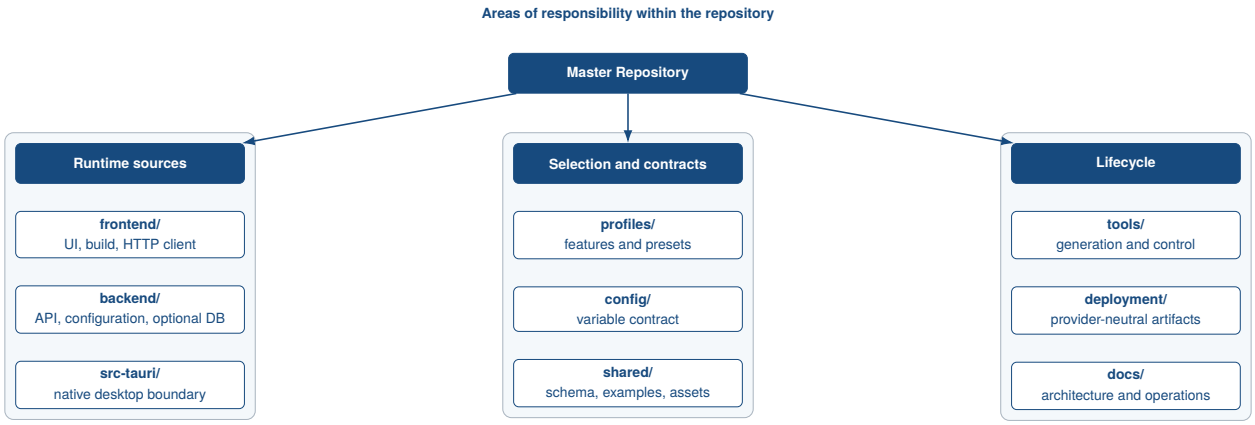

Figure 3: Components and responsibility boundaries in the Master Repository

Source: Author's own illustration based on kleiveist (2026f).

The separation of components in Figure 3 keeps the client, server, native integration, and shared artifacts distinct. Profiles and configuration describe the structure and runtime values; tooling and deployment control the components outside the product runtime.

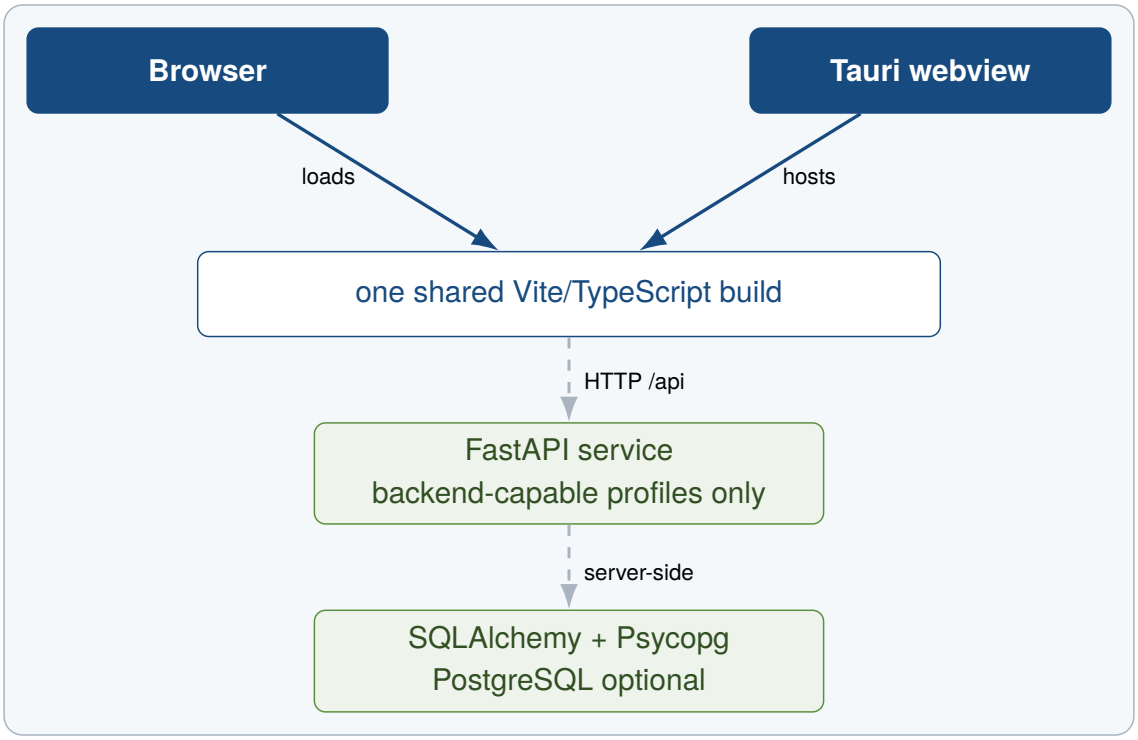

Figure 4: Profile-dependent runtime architecture

Source: Author's own illustration based on kleiveist (2026f) and kleiveist (2026g).

The browser or a Tauri webview loads the same Vite build (Figure 4). Backend-enabled profiles use FastAPI over HTTP; only the backend accesses the optional PostgreSQL persistence shown. Local product storage is separate and is not implemented (kleiveist, 2026h).

In simple terms: Reading the architecture diagrams

A software architecture is a blueprint showing components, their tasks, and their connections. A component is a distinct technical building block. A repository is a managed project directory with a history of changes; here, the master repository is the central template from which projects are generated. The client runs on the user's side and contains the visible frontend, while the server runs the backend in the background. A database stores data persistently. The optional PostgreSQL database shown here is accessed only by the backend. Deployment means providing and starting these components in a target environment. Separating responsibilities ensures that each component has a clear task and can be tested or modified as independently as possible.

3.2 Frontend with Vite and TypeScript

The frontend is the shared presentation layer for all five profiles. Vite is used for development and building; TypeScript checks the strictly configured source code, while Vitest tests the HTTP client (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

The generated profile module enables backend access only in appropriate variants. The browser and Tauri use the same source files and a public `VITE_API_BASE_URL`. Server logic, secrets, and direct database access remain outside the client; the web-only and desktop-local profiles require no backend

(kleiveist, 2026f, 2026g).

Table 3 documents the areas and versions in the commit under examination.

Table 3: Declared and resolved technology stack in the examined commit

Technology	Role in the template	Version / source	Project evidence
Vite	development server and frontend build	range from 6.0.7; lock: 6.4.2	`frontend/package.json`, `frontend/package-lock.json`
TypeScript	static checking of the frontend	range from 5.7.3; lock: 5.9.3	`frontend/package.json`, `frontend/tsconfig.json`
FastAPI	HTTP API and validation	$\geq$0.111, $<$1; production: 0.111.0	`backend/requirements.txt`, `backend/requirements-production.txt`
Tauri	desktop webview and packaging boundary	major version 2; lock: 2.11.5	`src-tauri/Cargo.toml`, `src-tauri/Cargo.lock`
Rust	native composition layer	at least 1.77; edition 2021	`src-tauri/Cargo.toml`
PostgreSQL	optional relational runtime unit	container: 16.4-alpine3.20	`deployment/compose.yaml`
SQLAlchemy	engine, session, and model foundation	$\geq$2, $<$3; production: 2.0.36	`backend/requirements-database*.txt`
Alembic	explicit schema migrations	$\geq$1.13, $<$2; production: 1.14.0	`backend/requirements-database*.txt`
Psycopg	PostgreSQL driver	$\geq$3.2, $<$4; production: 3.2.3	`backend/requirements-postgres*.txt`

Source: Author's own compilation based on the examined project state (kleiveist, 2026l).

In simple terms: Vite and TypeScript

The frontend is the visible, interactive side of the application. JavaScript is a language used for such applications; TypeScript extends JavaScript with type annotations that reveal certain errors during development. Vite is the tool that starts a development server and later produces the build. The development server makes the application available locally while it is being programmed, whereas the build creates the distributable files in `frontend/dist`. Vitest runs automated tests for frontend code. The HTTP client sends requests to a backend using a URL, which is its address on the network. In the template, this access is enabled only for profiles with a backend.

3.3 Backend with FastAPI

The backend forms a separate HTTP layer for the web-cloud, desktop-cloud, and full-platform profiles. Under `/api`, FastAPI currently provides only `/health` and `/ready`. Product-specific services, domain models, and authentication are not prescribed (kleiveist, 2026f; Ramírez, 2026).

Liveness reports the process state. When the database capability is active, readiness additionally checks the connection and responds with HTTP 503 on failure without disclosing internal details. The router, settings, and dependency are separately testable; service and domain modules are added only by the product.

The units remain separate in deployment (Figure 5): the Vite build can be deployed as static content, while FastAPI runs in its own unprivileged container. Compose represents this provider-neutral boundary locally; PostgreSQL is added only when explicitly selected (kleiveist, 2026e).

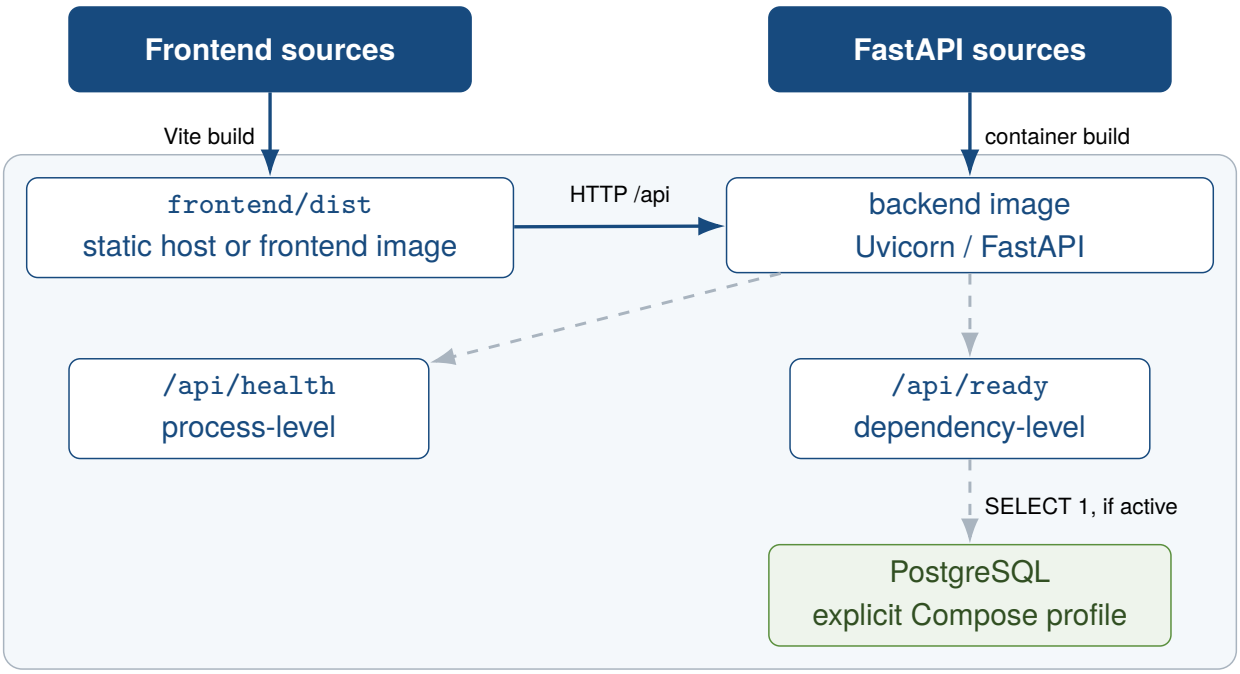

Figure 5: Provider-neutral deployment architecture

Source: Author's own illustration based on kleiveist (2026e).

In simple terms: FastAPI and HTTP

The backend is the part that operates in the background; here, it is built using the Python language and FastAPI. HTTP defines rules for how programs transmit requests and responses over a network. An API is an agreed interface through which the frontend communicates with the backend in this way. A route or endpoint is an address within this API that can be called, with `/api` grouping the shared check paths. As a liveness check, `/api/health` reports whether the process is running; as a readiness check, `/api/ready` reports whether it is ready for use and, when the database is enabled, also checks its connection. An HTTP status code summarizes the result as a number; here, HTTP 503 means that the service is temporarily not ready. A container packages a service together with its runtime as a separate unit, and the FastAPI backend runs in such a container during deployment. This baseline does not yet include product-specific services or authentication.

3.4 Desktop Integration with Tauri and Rust

Tauri provides a native boundary without duplicating the frontend. During development, it uses the Vite server; in the build, it uses `frontend/dist`. The Rust entry point only creates the Tauri application; native domain commands and a FastAPI sidecar are absent from the baseline (Klabnik et al., 2026; kleiveist, 2026f; Tauri Contributors, 2026b).

The default capability grants only `core:default`; a Content Security Policy restricts resources and development endpoints. Additional native permissions must be added on a product-specific basis (Tauri Contributors, 2026a). In the desktop-local profile, local functionality resides in the frontend or in future Tauri commands; desktop-cloud and full-platform use the public FastAPI URL. Packaging, signing, and the choice between a remote API, sidecar, or local commands remain product decisions.

In simple terms: Tauri and the desktop layer

Tauri uses the existing web frontend and displays it as a desktop application in a webview, an embedded area for displaying web content. An operating system such as Windows, macOS, or Linux controls the computer; here, Rust forms the native application layer, which can interact with the operating system directly and provide native functions. A native application is installed and runs on the computer rather than exclusively in a conventional browser. Together with permissions, a capability describes which native functions are allowed; the Content Security Policy (CSP) additionally restricts the content and connections that may be used. A sidecar is a separate helper program that can be distributed with the application, but the baseline does not provide one for FastAPI. A remote API, by contrast, runs on a remote server and is accessed over the network. Packaging bundles the application into installable packages, while signing cryptographically confirms their origin and integrity. Whether to use a remote API, sidecar, or local commands, as well as packaging and signing, remains a product decision here.

3.5 Shared Contracts and Interfaces

`shared/` is included in every generated project. It contains static assets, a JSON Schema conforming to Draft 2020-12, and valid and invalid test examples. The contents are serializable and framework-neutral, but define neither a product API nor executable framework code (kleiveist, 2026f, 2026l).

However, the runtime does not use the schema automatically: neither the frontend nor the backend imports it. The health contract is likewise mirrored manually as a TypeScript interface and a FastAPI response. Without code generation or synchronization, contract changes must be propagated on both sides and in consumer tests. The documented target role of shared contracts is therefore not yet fully implemented.

In simple terms: Shared data rules

An interface is a defined boundary at which two parts of a program exchange information. A contract describes the data format that both sides expect. JSON is a text-based format for structured data, and a JSON Schema defines machine-readable rules for its structure. Validation checks whether specific data complies with these rules. The frontend and backend must keep their formats synchronized because a one-sided change could otherwise break communication. Code generation can produce suitable code for both sides from a shared contract and thus simplify this coordination. In the state under examination, the schema and test examples exist but are not integrated automatically at runtime; automatic synchronization and code generation have also not yet been implemented.

3.6 Configuration Model

Project structure and runtime configuration are separate. The project manifest determines the features and is stored in `project-profile.toml`. The variable contract is defined in `config/environment.toml`. For active features, the priority order is CLI override, process environment, local `.env`, and contract default. Derived values also remain traceable (kleiveist, 2026j).

Figure 6 shows the security boundaries. Vite receives only the approved `VITE_API_BASE_URL`; FastAPI processes server values with explicit types and masks `DATABASE_URL`. The tooling forwards only relevant values and removes known secret names from the Tauri environment. Invalid or missing values result in controlled termination without exposing secrets.

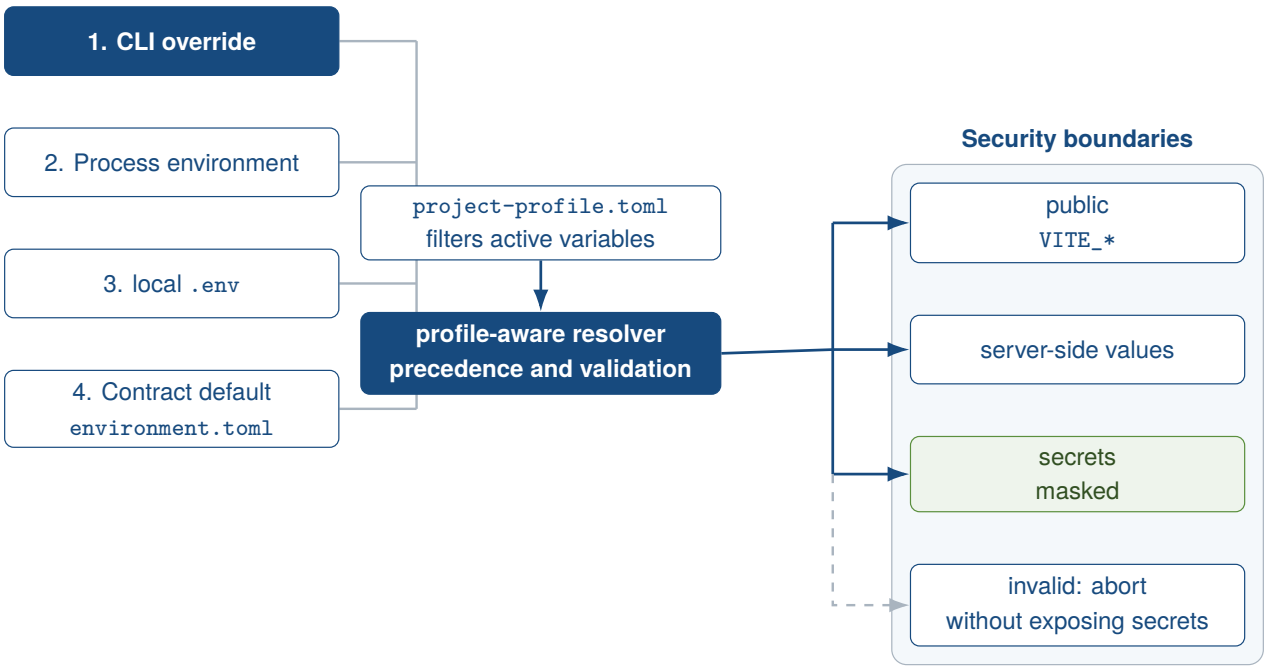

Figure 6: Configuration precedence and security boundaries

Source: Author's own illustration based on kleiveist (2026j).

In simple terms: Configuration

A configuration is a collection of settings that determines how and where a program runs. An environment variable is a value that the runtime environment provides to the program; a local `.env` file can collect such values for development. `environment.toml` describes the permitted variables and possible defaults, meaning the standard values to be used. `project-profile.toml`, by contrast, specifies which technical components belong to the project. An override deliberately replaces a lower-priority value, and the resolver determines the value that actually applies according to a fixed order. A secret is confidential information such as a password and must not reach the visible client. `DATABASE_URL` contains the server-side database connection and is masked. `VITE_API_BASE_URL`, by contrast, is the public backend address that the frontend may use.

3.7 Server-Side PostgreSQL Persistence with SQLAlchemy and Alembic

PostgreSQL is optional and is not a separate profile. `postgres` requires `database` and thus a backend; the web-only and desktop-local profiles are excluded. The database capability comprises the SQLAlchemy engine, sessions, and Alembic. PostgreSQL adds Psycopg 3, URL requirements, and integration tests and remains a separate runtime unit (kleiveist, 2026d; PostgreSQL Global Development Group, 2026).

The engine is created only upon access and validates connections before use; sessions are closed in a controlled manner. The empty declarative `Base` is an extension point for domain models (SQLAlchemy Authors and Contributors, 2026).

Alembic uses the same metadata basis for online and offline migrations. Without domain models, there is no initial revision. Neither `create_all()` nor automatic migrations at FastAPI startup are implemented; changes require explicit Alembic commands (Bayer, 2026; kleiveist, 2026d). `/api/health` remains independent of the database, while `/api/ready` checks reachability when the database ca-

pability is enabled.

In simple terms: The database layer

A database provides persistence: data therefore remains available after the program has been closed. A relational database organizes data into interconnected tables; SQL is the language used to query and modify them. PostgreSQL is this template's optional relational database. A connection is the communication channel to the database, and the Psycopg database driver provides it between Python and PostgreSQL. SQLAlchemy simplifies access: the engine manages the technical connection, while a session groups operations on data. The schema describes the database structure, such as tables and columns. A migration changes this structure in a controlled manner, and Alembic manages such changes. The baseline does not yet contain domain models or an initial migration; changes must be performed explicitly using Alembic commands.

3.8 Provider-Neutral Persistence Strategy for Product and User Data

The baseline separates template and development data, runtime configuration, and product and user data. `profiles/`, `project-profile.toml`, `tools/`, and `shared/` describe, generate, or verify the project. `.env`, `config/environment.toml`, `DATABASE_URL`, and API configuration instead determine how an instance runs. Documents, planning states, domain records, media, project files, and histories arise only through product use and require their own area of responsibility and lifecycle. Configuration files are not user-data storage; version-controlled template assets are not runtime assets of the finished product.

A persistence decision distinguishes provider-neutral local file storage, embedded databases, remote databases, and asset storage. Not every product requires all four classes. Local files may use JSON, Markdown, or product-specific formats; an embedded database could, for example, use SQLite. In contrast, the existing `database` and `postgres` capabilities implement server-side SQL persistence with SQLAlchemy, Alembic, and PostgreSQL. Asset storage for images, documents, graphics, or binary files remains separate from structured data. `local-files`, `embedded-database`, `sqlite`, `object-storage`, and synchronization are possible extension paths, not implemented capabilities (kleiveist, 2026h).

Each product project must identify the authoritative data source. In a local application, a local store may be the source of truth; in a server-centered application, it may be a central database. Offline or local-first systems additionally require explicit rules for synchronization direction, revisions, and conflicts; the baseline provides no synchronization engine. The product decision also covers the schema and versioning strategy, migration, backup and recovery, and security boundaries.

The template therefore standardizes neither the data itself, a universal format, nor a general storage provider. It standardizes the boundaries of responsibility and the decisions that must be documented for handling data. Storage format and persistence architecture, platform profile and storage provider, configuration and user data, and asset storage and structured data storage remain separate decisions.

In simple terms: Where user data is stored

Program files determine what the application can do. Settings determine how it runs. User data is created while people work with the product, such as documents, planning states, or images. Not every app needs the same storage. Small local applications may use files; more complex ones may need an embedded database. Server-based applications may use a central database, while images and documents may be stored separately. The template does not make this choice automatically. Each product must decide which store is authoritative and, if several stores exist, define rules for offline operation, synchronization, and conflicts. The baseline does not already include a local storage or synchronization solution.

4 Project Profiles and Feature Model

4.1 Basic Principle of the Profile Model

The master repository manages variants through features, thereby supporting systematic reuse without constituting a complete software product line (Clements & Northrop, 2001; Krueger, 1992). Here, *Core* denotes the paths included in every product project. A *Feature* groups the paths and dependencies of a technical capability, while a *profile* represents a permissible combination of features. A *capability* optionally extends a profile and is not itself a platform profile. The *manifest* records the selection in the generated project (kleiveist, 2026g, 2026l). The Core includes version information, configuration, documentation, profile descriptions, shared artifacts, and tooling, among other elements. Technical runtime components, by contrast, reside in the selected feature paths.

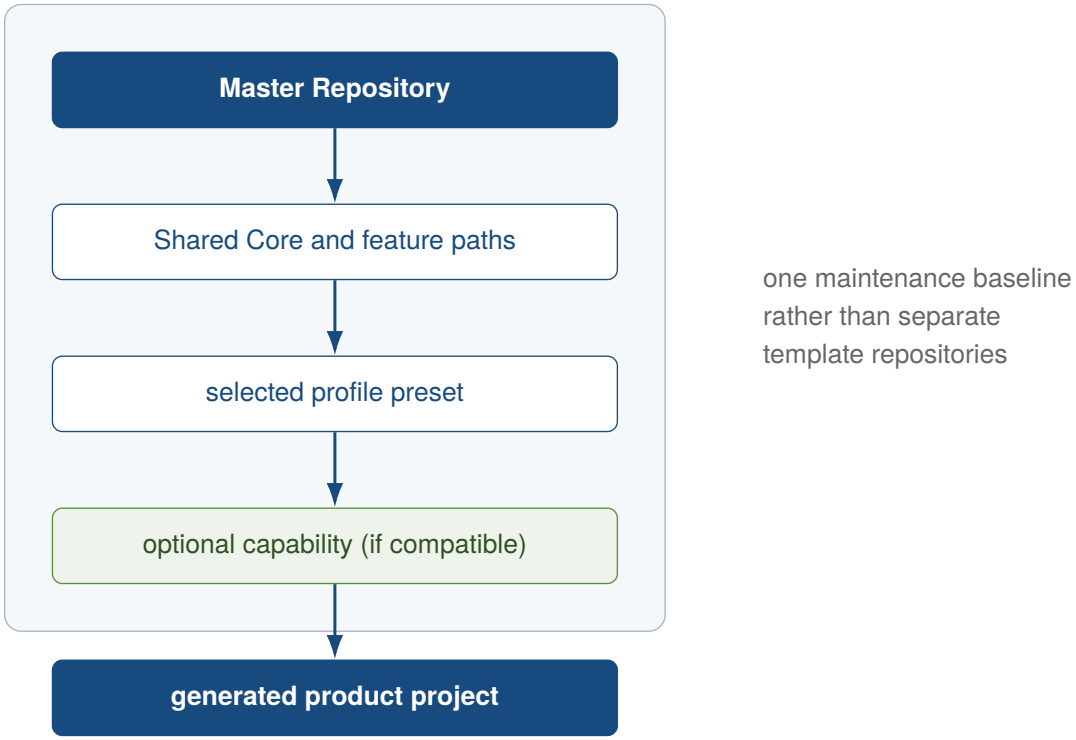

Figure 7: Deriving a product project from a shared template baseline

The generator resolves the profile and compatible capabilities and copies the Core and active feature paths into a new product project (Figure 7). Inactive directories are not selected; web profiles additionally omit the Tauri CLI. The name, slug, and, for Tauri, the application ID can be customized (kleiveist,

2026g, 2026m). Resolution produces an ordered scaffold plan. As a result, the target project does not merely contain disabled components; it predominantly contains only the directories intended for its variant.

Backend and cloud occur together in the current presets. For desktop projects, an additional browser channel distinguishes `full-platform` from `desktop-cloud`. Capabilities are checked only after the profile has been selected; the resolver does not silently add missing platform features (Figure 8).

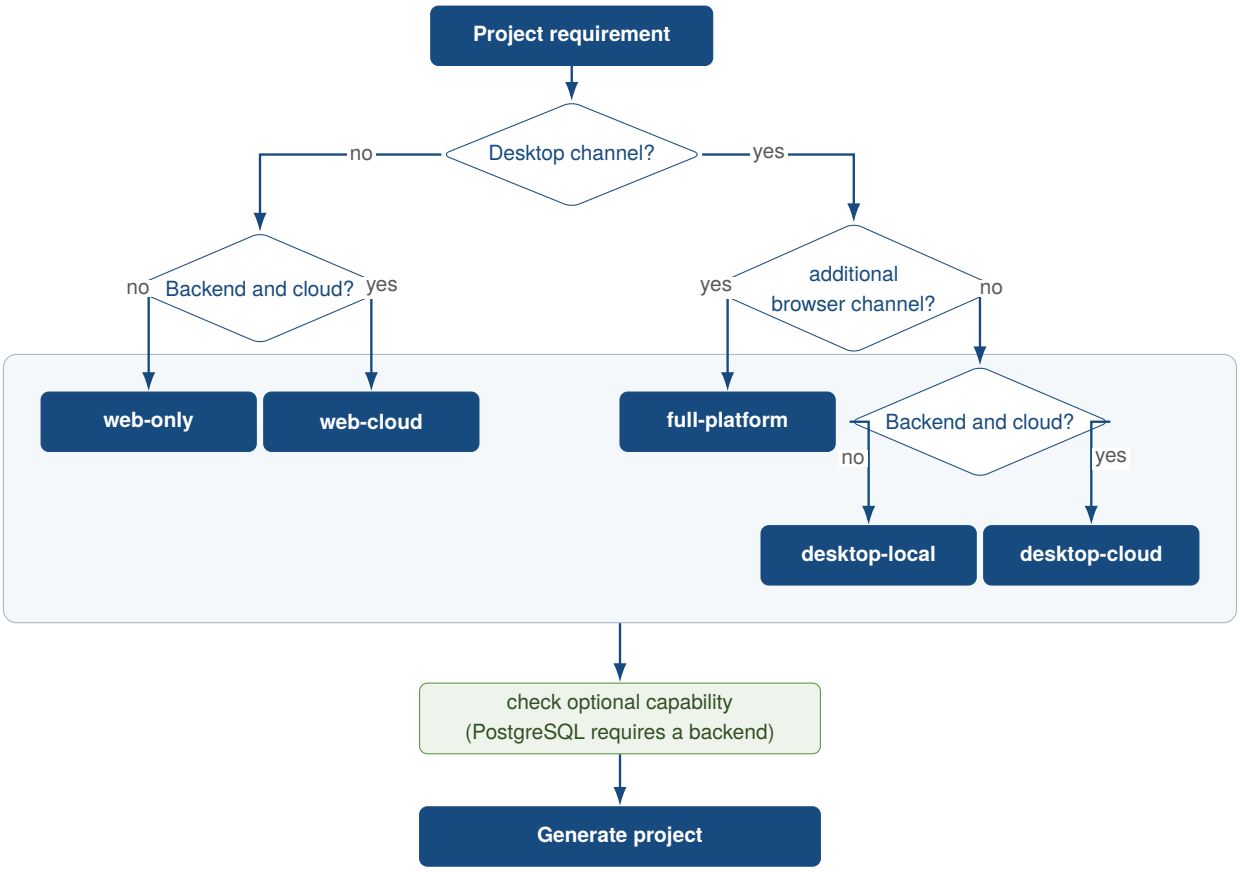

Figure 8: Decision flow for profile and subsequent capability selection

The generated `project-profile.toml` records the schema version, profile identity, requested extensions, and fully resolved features. The tooling validates it and handles only active services and workflows. The master manifest uses `full-platform + postgres` without modifying the preset itself. Runtime values, ports, URLs, and secrets instead belong to the separate configuration model (kleiveist, 2026j, 2026l). Requested optional features and the resolved result remain separately visible in the manifest. This distinction makes transitive dependencies traceable.

Figure 9 and Table 4 compare the five presets. None enables PostgreSQL by default.

Profile	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	included	–	–	–	incompatible
web-cloud	included	included	–	included	optional
desktop-local	included	–	included	–	incompatible
desktop-cloud	included	included	included	included	optional
full-platform	included	included	included	included	optional

Figure 9: Structural feature allocation of the five profile presets

Source: Author's own illustration based on the feature catalog and the profile definitions in the examined project state (kleiveist, 2026g, 2026l).

Table 4: Declared baseline features and resulting runtime directories

Profile	Baseline features	Runtime directories	PostgreSQL
`web-only`	`frontend`	`frontend/`	incompatible
`web-cloud`	`frontend`, `backend`, `cloud`	`frontend/`, `backend/`, `deployment/`	optional
`desktop-local`	`frontend`, `tauri`	`frontend/`, `src-tauri/`	incompatible
`desktop-cloud`	`frontend`, `backend`, `tauri`, `cloud`	`frontend/`, `backend/`, `src-tauri/`, `deployment/`	optional
`full-platform`	`frontend`, `backend`, `tauri`, `cloud`	`frontend/`, `backend/`, `src-tauri/`, `deployment/`	optional

In simple terms: Profile model

The *Core* is the shared foundation of every generated project. A *feature* is an individual technical component together with its files and required dependencies. A *profile* is like a preconfigured set of components; a *preset* is one such predefined selection. A *capability* adds an optional ability to a profile. The *manifest* records what was selected for a project and what was ultimately resolved. The *generator* creates the project, while the *resolver* first determines the permissible combination and its prerequisites. The resulting *scaffold* is the basic structure of required folders and files. A dependency is a prerequisite; a transitive dependency is an additional prerequisite needed only indirectly through another component.

4.2 Web-only Profile

`web-only` adds only the Vite frontend to the Core. The browser loads it without a FastAPI, desktop, deployment, or database layer; the Tauri script and CLI are removed. The profile is therefore suitable for pure client applications without backend access (kleiveist, 2026g). Compared with `web-cloud`, it consequently lacks the API and cloud feature; the generated frontend also does not enable a backend status request.

In simple terms: web-only

`web-only` is an application for the browser. It contains the Vite frontend. A dedicated FastAPI backend, a desktop layer, deployment, and a database are not part of this profile.

4.3 Web-cloud Profile

`web-cloud` combines a Vite frontend, a separate FastAPI backend, and provider-agnostic deployment files. It has no native desktop layer (kleiveist, 2026e, 2026g). The database module, Alembic, and Psycopg are added only with the compatible PostgreSQL capability, which must be selected separately. The frontend accesses the separate backend through a public URL. Without the capability, the profile remains database-free despite its cloud designation.

In simple terms: web-cloud

`web-cloud` is a browser application with a separate FastAPI backend. The profile also provides a provider-agnostic foundation for deployment. A PostgreSQL database is added only through the separately selected capability.

4.4 Desktop-local Profile

`desktop-local` runs the shared frontend in a Tauri webview. Backend, deployment, and database paths are absent (kleiveist, 2026g). Thus, *local* denotes the absence of a cloud API; no persistence provider is automatically part of the profile (kleiveist, 2026h). Because `database` requires a backend, PostgreSQL is incompatible. Local domain functionality must therefore reside in the frontend or in Tauri commands added for the specific product.

In simple terms: desktop-local

`desktop-local` runs the shared frontend as a Tauri application on the local computer. It does not include a central cloud API or a template backend. *Local* therefore does not automatically mean that a local database or another persistence provider is present.

4.5 Desktop-cloud Profile

`desktop-cloud` connects the Tauri frontend to the separate FastAPI backend through a public URL and adds deployment files. PostgreSQL is compatible but remains optional; neither a local store nor PostgreSQL is established as the source of truth (kleiveist, 2026e, 2026g, 2026h).

The technical base features correspond to `full-platform`; the profile ID, however, identifies the desktop client as the intended product channel. The distinction is semantic: it does not introduce an additional technical component.

In simple terms: desktop-cloud

`desktop-cloud` is a Tauri desktop application with a central FastAPI backend. The desktop client reaches this backend through a public URL. PostgreSQL can be used, but it remains an optional extension.

4.6 Full-platform Profile

`full-platform` provides the browser and Tauri as two product channels for the same frontend base; both access the same FastAPI backend. Its scaffold differs from `desktop-cloud` only in profile identity and description, not in its base features (kleiveist, 2026g, 2026l). The browser build and desktop application thus use the same frontend and backend base.

Full does not include a persistence provider; `postgres` is the selectable server-side SQL extension (kleiveist, 2026h).

In simple terms: full-platform

`full-platform` offers the browser and desktop as two ways to access the same technical base of frontend and backend. Both use the same FastAPI backend. Despite the name, a database is not included automatically. `postgres` is the selectable server-side database extension; other storage forms remain product decisions.

4.7 Optional Capabilities

The catalog contains six features. `tauri` requires `frontend`; `cloud` and `database` require `backend`; and `postgres` requires `database`. The directed rules in Figure 10 do not describe runtime communication (kleiveist, 2026d, 2026g).

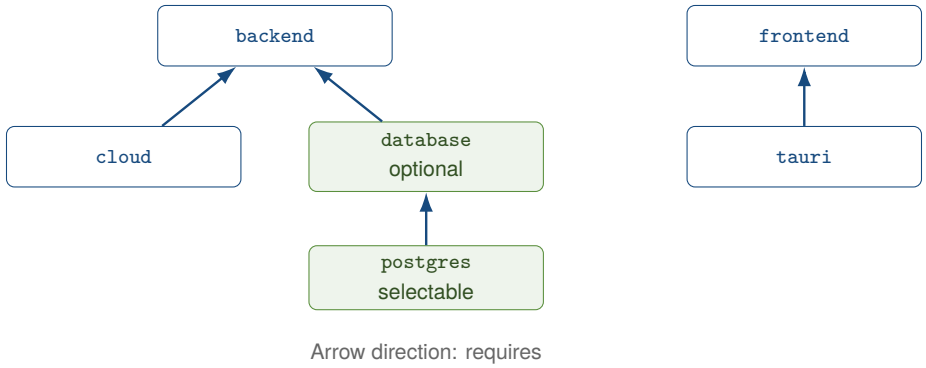

Figure 10: Implemented dependencies in the feature catalog

Only `postgres` is currently marked as a selectable capability. For backend profiles, it first resolves the optional, internal-only `database` feature and adds SQLAlchemy, Alembic, Psycopg, and tests. The PostgreSQL service is already declared as an inactive profile in the Compose document; the capability adds backend infrastructure and dependencies (kleiveist, 2026e, 2026l). Thus, `web-cloud + postgres` consists of the frontend, backend, cloud, database base, and PostgreSQL extension. The selection therefore does not merely activate the Compose entry; it adds the associated backend paths.

The resolver adds only optional prerequisites. It therefore rejects `web-only + postgres` and `desktop-local + postgres` because both lack a backend. Declaring new catalog entries alone does not make them implemented. They also require their own paths, dependencies, and tests.

Two implementation discrepancies remain: the dialog offers only `selectable` PostgreSQL, but a direct `--with database` argument accepts the database capability that is optional for backend profiles. In addition, contrary to the general profile documentation, entries for `.env.example` originate from `conf`

`ig/environment.toml`, not from `features.toml`. The presets remain unchanged, but the distinction between internal and selectable capabilities is not fully enforced.

In simple terms: Optional capabilities

Features can be prerequisites for other features; for example, Tauri requires the frontend. An optional capability is added only when requested, and *selectable* means that it is normally offered as a choice. Currently, only PostgreSQL can be selected in this way. PostgreSQL requires the database base, which in turn requires a backend because that is where database access runs. Therefore, `web-only + postgres` and `desktop-local + postgres` are not permitted. The resolver checks such dependencies, but it cannot add a missing platform feature on its own.

4.8 Single-Repository Strategy

The five profiles are not template forks. The Core, feature implementations, profiles, and generator reside in a single maintenance base from which independent product projects are created (kleiveist, 2026g, 2026l).

The master also contains optional implementations and itself enables PostgreSQL. A generated project, by contrast, receives only the Core, resolved feature paths, manifest, frontend profile, and an appropriate `.env.example`. It then remains separate from the unchanged master. Centralized maintenance reduces parallel baselines; generated projects are not automatically synchronized back to the master. This separation protects the maintenance base from product changes. Conversely, existing generated products do not benefit from subsequent fixes to the master unless those fixes are incorporated deliberately.

In simple terms: One shared maintenance base

Single repository means here that the Core, features, profiles, and generator are maintained together in one place. This master repository is the authoritative *source of truth*, meaning the reliable baseline for the template. The profiles are not separate forks or copies of this maintenance base. The generator uses it to create an independent product repository with the selected parts. The master and product project then remain separate. Later updates to the master are not synchronized automatically; they must be incorporated into the product deliberately.

5 Tooling and Development Workflow

5.1 Role of control.py

`tools/control.py` consolidates the public workflows. Its commands cover initialization, diagnostics, installation, development, configuration, database operations, tests, builds, containers, Tauri, versioning, releases, and documentation. Help and group views structure this interface (kleiveist, 2026a, 2026m). The principal entry points include `init`, `doctor`, `install`, `run`, `test`, and `build`. The `db`, `tauri`, `container`, and `release` groups add only the specialized workflows required in each case.

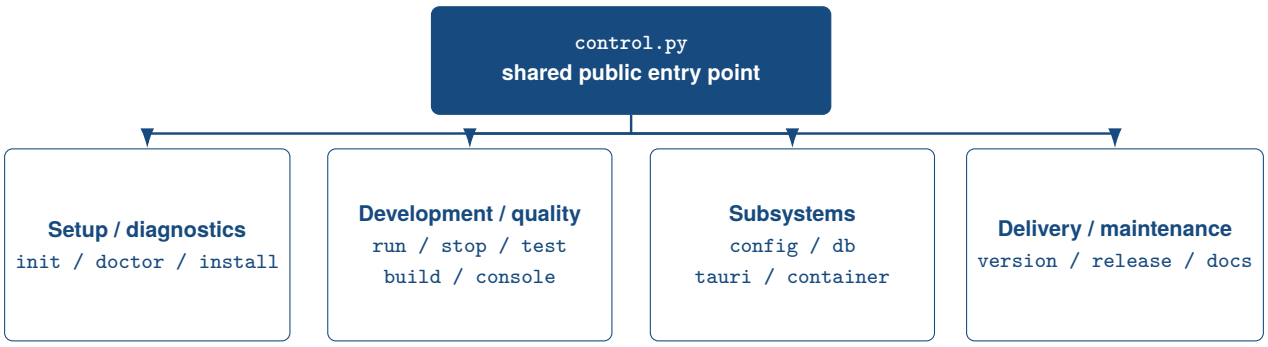

Figure 11: Command groups of the central project tooling

Source: Author's own illustration based on kleiveist (2026a) and kleiveist (2026m).

The handlers load the manifest and, when required, the runtime configuration, then delegate to the respective specialized tools. These tools remain independent. The guided `console` invokes the same commands and does not form a second system.

Different exit codes distinguish help and success, domain errors, usage errors, and cancellation. Expected profile and generation errors are shown without a traceback; unexpected exceptions are logged. This convention supports interactive diagnostics and CI evaluation (kleiveist, 2026a, 2026c). In particular, successful execution, domain failure, and incorrect usage can be distinguished programmatically. Warnings do not automatically block the workflow.

In simple terms: control.py

`tools/control.py` is a Python script that serves as the template's command center. A terminal is a text-based window in which programs are controlled through typed input. In the terminal, the script provides a *command-line interface* (CLI). A command such as `run` selects a task; an argument supplies additional information for that task. *Tooling* refers to helper tools for workflows such as installation, testing, and building. An exit code is a number through which the script reports success, a domain error, or incorrect usage. For an unexpected Python error, a traceback shows the sequence of code locations involved; expected usage and profile errors are displayed without such a detailed technical report.

5.2 Project Initialization and Generation

`init` creates an independent product project from a profile and compatible capabilities. Selection can be guided or non-interactive; project identity, target, and dry run are configurable. A modified Tauri identity requires a valid reverse-domain identifier (kleiveist, 2026g, 2026m).

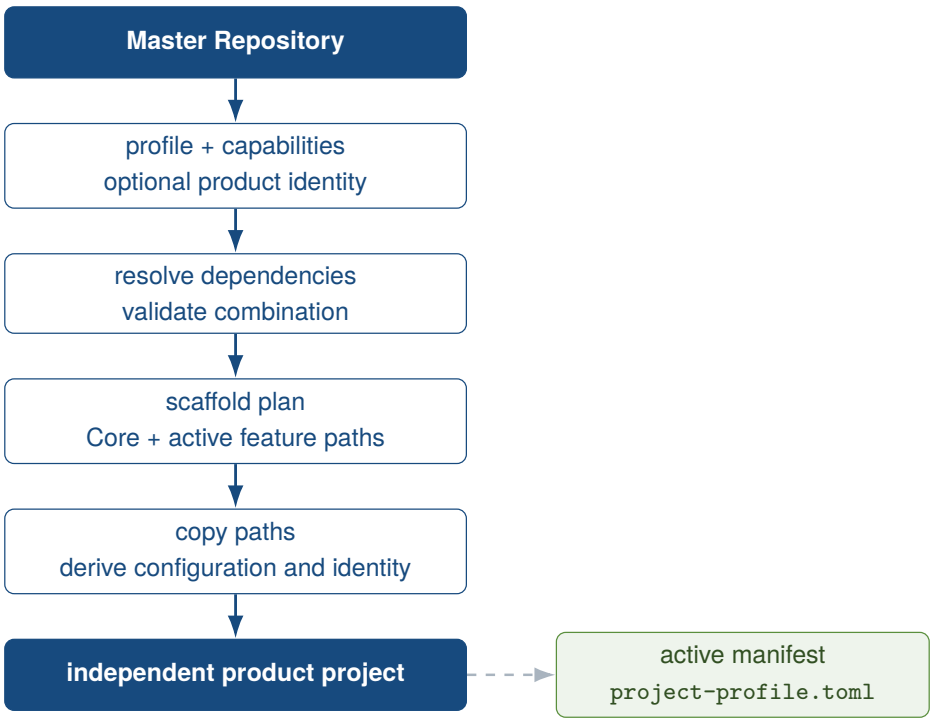

Figure 12: Workflow for profile-driven project generation

Source: Author's own illustration based on kleiveist (2026g) and kleiveist (2026m).

The generator resolves dependencies, creates the scaffold plan, and validates the sources and target before writing. It copies the Core and active features, generates the manifest, frontend profile, and `.env.example`, and adjusts existing product metadata. Transient directories are omitted; web profiles omit the Tauri script and CLI. When a product identity is specified, only frontend, backend, Compose, Tauri, and Cargo metadata that are actually present are adjusted. The scope thus remains dependent on the selected profile.

Within the master, targets are permitted only under `.generated/`; external targets are also possible. The repository itself and nonempty targets are rejected, and `--dry-run` writes nothing. One limitation remains: only the dialog filters by `selectable`; a direct `--with database` argument is accepted for backend profiles (kleiveist, 2026l). These preliminary checks keep the master and target separate and prevent partially generated projects in known conflict cases. The derivation is designed to be deterministic; nevertheless, the empirical evidence of reproducibility remains limited to the case examined.

In simple terms: Generating a project

The `init` command starts the generator for a new product project. The generator builds a *scaffold* from the selected profile: a prepared basic structure of folders and files. The target directory is the location where this project will be created. A *dry run* checks the intended process without writing files. Project identity includes details such as the name and short name that distinguish the generated project from the template. An application ID uniquely identifies a Tauri application. A *reverse-domain identifier* is such an ID in which words, as in `com.example.product`, are arranged from the general Internet domain to the specific product.

5.3 System Check and Installation

The read-only `doctor` checks runtimes, configuration, paths, dependencies, and ports according to the profile. Disabled layers are not an error. In general, missing Docker produces only a warning; container, database, and Tauri prerequisites have their own stricter diagnostic paths (kleiveist, 2026j, 2026m).

`install` sets up only active components: frontend dependencies are preferentially installed from the lockfile, and backend dependencies are installed in a virtual environment. Database packages, the tooling environment, and Chromium are added only when required; for Python, a pip/venv fallback is available alongside `uv`. A local `.env` remains unchanged. `tauri install` handles native operating-system and Rust prerequisites separately. Specifically, the frontend uses `npm ci` when a lockfile is present. An active backend receives its own environment and only the profile-dependent requirement sets. Chromium is installed only when Playwright is configured; disabled layers are skipped.

The README requires Git, Python 3.11 or later, Node.js 20 or later, and Rust Stable for desktop profiles. The general Doctor does not check Git or these version ranges; the Tauri Doctor accepts any active Rust toolchain. CI, by contrast, explicitly sets up the specified major versions of Python and Node. Local enforcement therefore differs from the documentation (kleiveist, 2026c, 2026l). This limitation concerns how the prerequisite is checked, not whether the documented prerequisite applies.

In simple terms: Prerequisites and installation

`doctor` performs read-only checks to determine whether the runtimes, settings, paths, dependencies, and ports important to the selected profile are available. Installation sets up required dependencies: additional programs and libraries on which the project relies. A *package manager* automates the downloading and setup of such packages. Node.js runs the frontend tooling; npm manages its packages, and `npm ci` installs the versions specified in the lockfile. Python runs the backend and tooling; a virtual environment (*venv*) separates their packages from the rest of the system, while `uv`, or `pip` as a fallback, installs them. Git manages source-code changes and is required to clone the repository. Rust is required for the native Tauri layer of desktop profiles. Chromium is the browser used for configured Playwright tests and is set up only when needed; other optional tools are likewise handled according to the profile.

5.4 Development Mode

Figure 13 shows the profile-dependent path from diagnostics and installation to quality assurance and delivery.

`run` starts Vite and, only when the backend is active, Uvicorn as well. In the foreground, cancellation terminates the tracked process groups; in detached mode, PID and log data are stored. `stop` terminates tracked processes and, by default, listeners on the configured ports as well; `--tracked-only` restricts this behavior to the tool's own tracked state (kleiveist, 2026a, 2026m). In the foreground, output from active services is combined. Detached mode, by contrast, retains process and log data under `tools/.runtime`; this state store provides the basis for `stop`.

The separate desktop path delegates to `tauri dev` and removes known server-side secrets from the child-process environment. It supports foreground and background operation. The console merely guides users through the same CLI workflows. The native development path thus remains separate from general Vite/Uvicorn operation without duplicating the command logic.

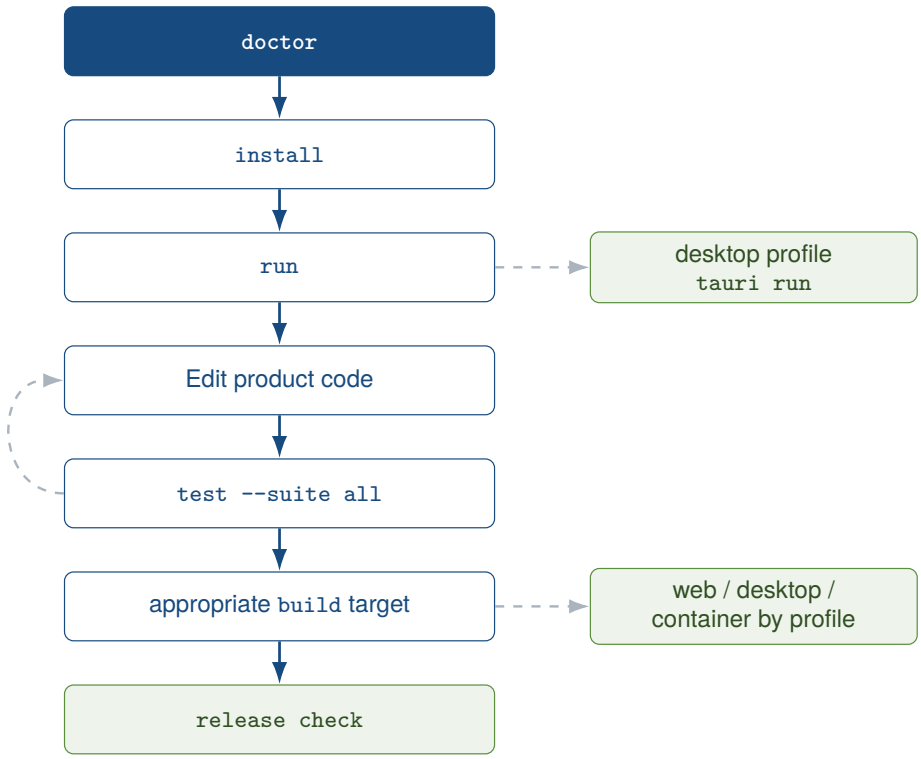

Figure 13: Basic structure of the development workflow
Source: Author's own illustration based on kleiveist (2026m) and kleiveist (2026i).

In simple terms: Programs during development

A development server makes an unreleased application accessible on the local computer while it is being developed. For this purpose, `run` starts Vite and, when the backend is active, Uvicorn as well. Vite serves the frontend, and Uvicorn serves the FastAPI backend. A running program is called a process; a port is its numbered access point for connections. In the foreground, output remains directly visible in the terminal, and cancellation terminates the tracked processes. In the background, they continue running in detached mode. A PID identifies a process, while a log records its messages. `stop` terminates the tracked processes and, depending on the option, other listeners on their configured ports as well.

5.5 Tests and Builds

The eight test suites cover tooling, schema, API, database, PostgreSQL, frontend, E2E, and Tauri/Rust. `test --suite all` provides the complete project-wide entry point; reports are optional (kleiveist, 2026a, 2026m). Among other elements, the tooling suite tests the generator, profiles, and configuration; the component suites delegate to Pytest, Vitest, Playwright, or Cargo. The test areas therefore remain independently executable.

Selection remains profile-dependent: if a feature is absent, the suite reports `SKIP`; if sources or prerequisites are missing for an active feature, it reports `FAIL`. PostgreSQL without a test URL and E2E without Playwright configuration are skipped. The runner can repair missing backend environments and manage the development services for E2E. This behavior deliberately distinguishes absent features from defective active components. A skipped test is therefore not evidence of success.

The build paths remain separate: web produces Vite output and a ZIP; desktop delegates the target and bundle to Tauri; and container builds frontend, backend, or both images for cloud profiles.

`container validate` checks the Compose model without starting it (kleiveist, 2026b, 2026i). The web path additionally creates a ZIP package. Desktop builds remain tied to an active Tauri feature and the respective target platform.

GitHub Actions uses the same public entry points for the Core, profile matrices, PostgreSQL, and desktop targets. CI supplements these with operating systems, runtimes, and a temporary PostgreSQL service (kleiveist, 2026c). Local and external workflows therefore use comparable entry points, but do not run under identical environmental conditions.

In simple terms: Tests and builds

An automated test performs predefined checks without requiring a person to click through them manually. A unit test examines a small, self-contained part of the code. An integration test checks whether multiple parts, such as the backend and database, work together. An E2E test follows a complete process from a user's perspective. A test suite groups related tests; the template uses Pytest for Python, Vitest in the frontend, Playwright for E2E, and Cargo for Rust. *PASS* means passed, *FAIL* means failed, and *SKIP* means skipped; a skipped test is not evidence of success. A build processes the source code into a deliverable artifact, such as web output or a desktop package, depending on the path. A ZIP groups files in an archive; a container image packages a runtime environment from which a container can be started.

5.6 Release and Packaging Workflow

`VERSION` is the central version source; validation and write-based synchronization are separate. `release check` does not produce artifacts. The command checks the version, product identity, Tauri rules, tag, and Git working tree. Inconsistencies, placeholders from the template, and a mismatched or modified state are errors. Missing signing configuration, by contrast, produces only a warning. Only the master repository accepts its canonical template identity (kleiveist, 2026a, 2026i).

Web delivers a Vite build and ZIP. On the respective host, Tauri produces native Windows, macOS, or Linux bundles; additional paths are available for a portable Windows ZIP and a Linux cross-build. CI produces only ephemeral, unsigned validation artifacts. These packages validate the technical packaging path. Without a signature and product-specific approval, they are not publishable releases.

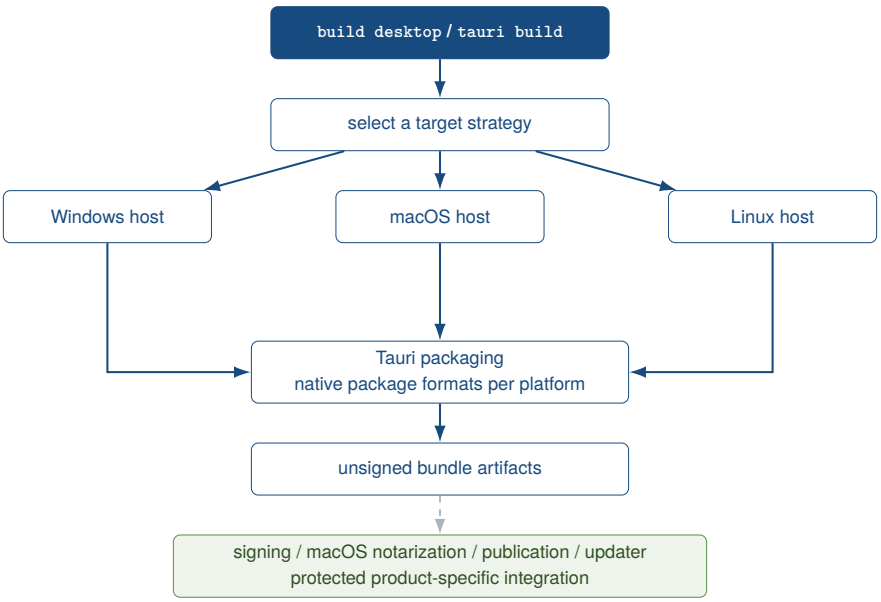

Figure 14: Cross-platform model for desktop releases

Source: Author's own illustration based on kleiveist (2026i).

Signing and notarization are product-specific. The same applies to publication, app stores, and updaters. The release workflow combines tests with web and container builds. These are followed by the gate and the unsigned desktop matrix. The workflow neither publishes nor deploys (kleiveist, 2026c, 2026i).

For cloud profiles, the container commands check Docker, Compose, and deployment files and build versioned images. Compose simulates production locally and optionally runs PostgreSQL. The foundation does not prescribe a provider and replaces neither secrets management nor a controlled migration job (kleiveist, 2026b, 2026e). A Docker/Compose foundation is therefore not equivalent to a complete architecture for AWS, Azure, or GCP.

In simple terms: From version to package

A version is a unique designation for a particular state; in the template, `VERSION` is the central source for it. A release is a version approved for publication, but the workflow examined here does not itself publish anything. `release check` merely checks the prerequisites and produces no artifacts. During *packaging*, program files are combined into a deliverable package or *bundle*; this result is an artifact. A Git tag marks a particular commit with a name, often the version number. During *signing*, a digital signature backed by a certificate confirms a package's origin; notarization is an additional external check, and an updater distributes later versions to installed applications. Deployment means making an application available; Docker builds images for this purpose, containers run them, and Docker Compose describes multiple related containers. AWS, Azure, and GCP are possible cloud providers, but this provider-agnostic foundation does not automatically commit to any of them.

5.7 Developer Onboarding

Onboarding follows the sequence of cloning the master, running `doctor`, running `init`, moving to the target project, running `doctor` again, and then running `install` and `run`. Product code, configuration, tests, and commits then belong in the derived repository. Template maintenance, by contrast, takes

place in the master and validates the entire baseline there (kleiveist, 2026g, 2026l). This separation prevents product development from inadvertently modifying the shared template.

The basic prerequisites are Git, Python 3.11 or later, and Node.js 20 or later. Desktop profiles additionally require Rust Stable and platform-specific Tauri packages; Docker, PostgreSQL, and PyGitIndex are required only for the respective optional tasks. Optional tools therefore do not block every profile.

The README and specialized documents organize getting started, tooling, profiles, configuration, CI, containers, and releases into separate areas. This standardizes the public setup paths, but does not yet demonstrate a measurable reduction in onboarding time (kleiveist, 2026c, 2026i, 2026j, 2026m). The documentation structure thus supplements technical onboarding, but does not replace an empirical measurement of its duration.

In simple terms: Getting started for developers

Onboarding is the process through which a developer sets up the project and its workflows for the first time. Cloning a repository means using Git to create a local working copy of it. The development environment comprises the required programs, dependencies, and settings on the developer's computer. The master repository is the jointly maintained template; the actual product code then belongs in the separate product repository. The sequence is: clone the master → `doctor` → `init` → target project → `doctor` again → `install` → `run`. Thus, the template is checked first, after which the generated target project is set up and started.

6 Quality Assurance and Verification

6.1 Test Strategy

Verification associates every observation with an artifact, an executed check, and a commit-bound result. Test code alone does not demonstrate execution, just as a workflow file does not demonstrate a successful CI run. Execution results and the acceptance report therefore carry more weight than code, configuration, and documentation (ISO/IEC, 2023; Runeson & Höst, 2009).

Four levels cover the baseline: tooling tests check the CLI, profiles, generator, configuration, release, and repository; component tests cover the schema, backend, frontend, and Tauri/Rust. Integration tests exercise PostgreSQL 16 together with the configuration and Alembic. Build checks produce web, container, or native artifacts (kleiveist, 2026c).

The scope follows the active profile. *PASS* denotes a successful check, and *FAIL* an executed check that failed. *Not verified* means that evidence is absent, while *out of scope* denotes a deliberate boundary; disabled items are not applicable. Generation and structure checks address A-01 and A-03, the public tooling interface addresses A-04, and valid and rejected capability combinations address A-07. Figure 15 classifies this evidence.

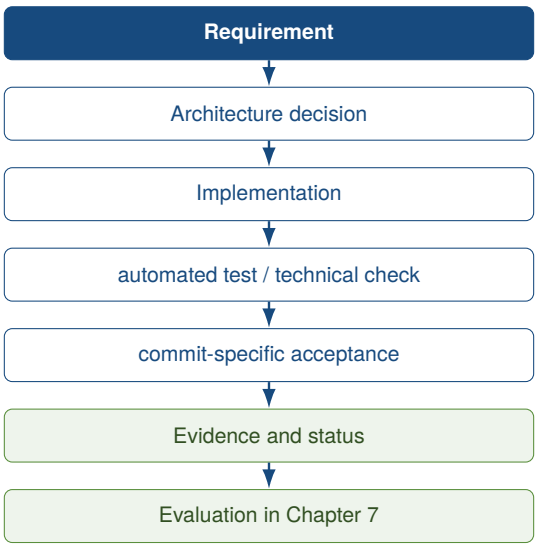

Figure 15: Evidence chain for technical verification

Source: Author's own illustration.

In simple terms: Quality and evidence

Quality assurance comprises the measures used to find errors and check requirements. A test is a specific check that has been executed. Verification here means checking in a traceable way whether expected technical behavior is actually supported by evidence. Evidence is a verifiable result; an artifact may, for example, be a generated package or a report. A commit is a uniquely recorded state of the source code, so evidence bound to a commit applies only to that exact state. *PASS* means passed, *FAIL* means executed and failed, and *SKIP* means not executed or not applicable. Missing evidence therefore means *not verified*; it is neither a success nor automatically an error.

6.2 Continuous Integration

Core CI, Profile Matrix, PostgreSQL Integration, and Desktop CI separately check the baseline, five profiles, PostgreSQL combinations, and native packages. Release Validation runs only manually or for version tags. All workflows have read permissions and use `tools/control.py` (kleiveist, 2026c).

The runs dated 7 August and queried on 9 August 2026 correspond to HEAD `a4487ac` and are not entirely green. Core CI succeeded for the backend, schema, frontend, web build, Compose, and both images; tooling, profiles, and configuration failed. In the profile matrix, both web profiles passed, while the three desktop profiles failed during the project test. PostgreSQL integration succeeded for the master and web-cloud, but not for the two desktop combinations. Desktop CI packaged on Linux and macOS; Windows failed during frontend installation (GitHub, 2026a, 2026c, 2026d, 2026e).

Locally, on 9 August 2026, all 194 tooling tests as well as the schema, API, and SQLAlchemy tests passed on the same HEAD under Python 3.13.14. These local results neither supersede the failed external jobs nor provide evidence for the paths that were not executed because Node.js, Rust, and Docker were unavailable. Release Validation was not run for this commit.

In simple terms: Continuous Integration

Continuous Integration (CI) means that checks are run automatically on a separate system after source-code changes. In the project examined, GitHub Actions provides this service. A *workflow* describes such an automated process. Within it, a *job* is an individual group of steps that runs on a *runner*, meaning the computer provided for that purpose. A *pipeline* is the complete chain of these automated checks. A commit denotes a fixed code state; *HEAD* points to the commit currently relevant to the working copy or branch under consideration. A green CI run has passed all required jobs, while a red run contains at least one failure. The runs for commit `a4487ac` evaluated in the main text are not entirely green, even though certain local checks passed.

6.3 Acceptance and Technical Verification

The final acceptance report rates commit `1cdfb6e` as *PASS WITH OPEN EXTERNAL ITEMS* and *READY FOR CASE STUDY*. This applies to the case study baseline, not to production readiness or external deployments. On this commit, 189 tooling tests passed (kleiveist, 2026k).

The acceptance process generated and installed all five profiles, executed profile-dependent diagnostics, tests, and web builds, and packaged the three desktop profiles as Linux DEBs. With Compose available, strict validation passed. The three backend profiles additionally passed with PostgreSQL, including the connection, migrations, tests, and build; the desktop variants were packaged again. As expected, the two incompatible PostgreSQL combinations were rejected before the target was generated (kleiveist, 2026k).

Figure 16 and Table 5 distinguish local acceptance from current CI. The more recent failures do not invalidate the commit-bound acceptance, but they prevent its application to `a4487ac` as fully green remote evidence.

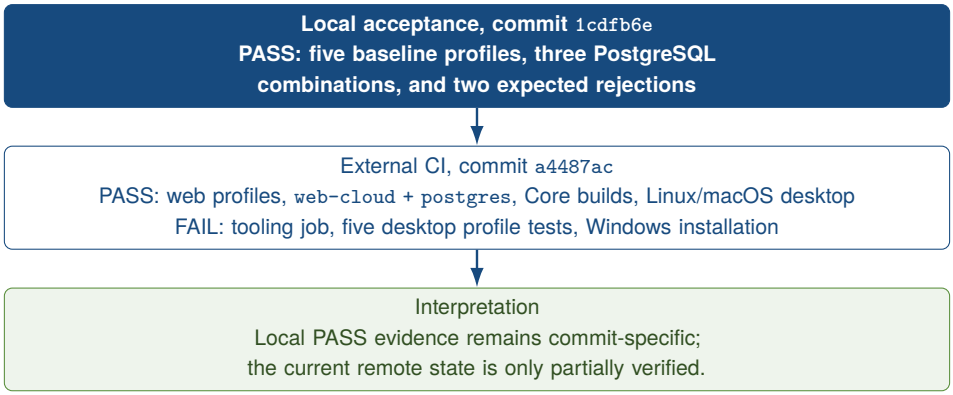

Figure 16: Profile verification by evidence state and commit

Source: Author's own illustration based on kleiveist (2026k) and the current GitHub Actions runs.

Table 5: Profile-specific technical verification

Profile / combination	Gen.	Install / Doctor	Tests	Web	Desktop	PG / migr.	CI on `a4487ac`
`web-only`	P	P	P	P	–	–	P
`web-cloud`	P	H	P	P	–	–	P
`desktop-local`	P	P	P	P	P (Linux DEB)	–	F: profile test
`desktop-cloud`	P	H	P	P	P (Linux DEB)	–	F: profile test
`full-platform`	P	H	P	P	P (Linux DEB)	–	F: profile test
`web-cloud + postgres`	P	P	P	P	–	P	P
`desktop-cloud + postgres`	P	P	P	P	P (Linux DEB)	P	F: profile test
`full-platform + postgres`	P	P	P	P	P (Linux DEB)	P	F: profile test

Legend: P = PASS; H = PASS WITH NOTE; F = executed check failed; – = not applicable. The six central result columns are taken from the local acceptance on `1cdfb6e`; the final column reports the profile-specific GitHub Actions run on `a4487ac`. `web-only + postgres` and `desktop-local + postgres` were correctly rejected before target generation in the local acceptance (P). Source: kleiveist (2026k) and GitHub (2026d, 2026e).

In simple terms: Technical acceptance

Technical acceptance uses defined steps to check whether a software base meets the intended technical requirements. Local evidence is produced on a working computer; remote evidence may come from an external CI system. Commit-bound means that the result applies only to the source-code state checked at that time. Acceptance for `1cdfb6e` passed with open external items and confirms its suitability as the case study baseline. It does not turn later failures on `a4487ac` into passed remote checks. Production readiness would additionally demonstrate that a product can be operated safely and reliably in real-world use. The acceptance expressly does not confirm such production readiness or an external deployment.

6.4 Reproducibility

Two generations of `desktop-cloud + postgres` on `1cdfb6e` were byte-for-byte identical for identical inputs. `init --dry-run` wrote nothing; nonempty targets and incompatible capabilities were rejected before writing. This evidence applies only to this combination and environment (kleiveist, 2026k; Lamb & Zacchiroli, 2022).

Frontend, Cargo, and three Python lockfiles pin dependencies; npm and Cargo use locked versions, and the container baseline additionally uses hashes. The hash-verified installation process was tested (kleiveist, 2026b, 2026c). The operating system, toolchain, registries, network, and platform SDKs nevertheless limit reproduction; native results remain platform-specific.

In simple terms: Reproducible results

A reproducible build should produce the same result again from the same inputs. For this case study, however, the evidence only shows that two generations of `desktop-cloud + postgres` on `1cdfb6e` were byte-for-byte identical in the same environment. A pinned dependency version specifies exactly which version of a required library is used. A lockfile pins such versions instead of selecting arbitrary newer packages on the next run. A hash is a short digital fingerprint that can verify specific content very reliably. The toolchain comprises the runtimes, compilers, and build tools in use. Differences in the operating system, toolchain, network, or platform SDKs can nevertheless affect reproduction.

6.5 Security and Configuration Boundaries

Tooling tests provide evidence for configuration precedence and for validation of environments, ports, and CORS origins. They also check the profile-dependent `.env.example` and ensure that `DATABASE_URL` is present only for PostgreSQL. These tests passed locally, while the external tooling job failed on the same commit (GitHub, 2026a; kleiveist, 2026j).

Only `VITE_API_BASE_URL` may reach the frontend. Tests reject secrets designated as public, remove `DATABASE_URL` from the frontend and Tauri, and mask database passwords in diagnostics. No database variable is created without PostgreSQL. Additional checks cover health, readiness, error responses, database-independent liveness, the Tauri CSP, and capability metadata (kleiveist, 2026i, 2026k).

Thus, only the configuration, secret, and desktop boundaries of the baseline have been verified. Authentication, authorization, production credentials, TLS, and provider-specific hardening remain product-specific; a complete security assessment has not been demonstrated.

In simple terms: Security boundaries

Security means protecting an application, its data, and access to it from unauthorized use. A *secret* is a confidential value; *credentials* are data through which a person or service proves its identity. CORS defines the web origins from which a browser may permit requests; a CSP limits the content and actions an application may load or execute. TLS encrypts transmission over a network. Authentication answers the question *Who are you?*; authorization answers *What are you allowed to do?* RBAC implements such permissions through assigned roles. Only certain configuration, secret, and desktop boundaries of the baseline are verified here, while authentication, authorization, production credentials, TLS, and a complete security assessment remain the responsibility of the specific product.

6.6 Outstanding External Checks

Table 6 distinguishes failed checks, missing evidence, and deliberate product boundaries. Failed CI jobs are real errors; a Compose stack that was not started, by contrast, is not verified.

The current Core job provides evidence for Compose validation and both master images, but not for either `docker compose up` or production deployment. Linux and macOS packages were built unsigned on `a4487ac`; the Windows package was not. Branch Protection could not be read because the required API permission was unavailable. Signing, notarization, updaters, production credentials, and authentication/RBAC require a product and external infrastructure (GitHub, 2026a, 2026c; kleiveist, 2026i).

Table 6: Outstanding and external verification items on `a4487ac`

Verification item	Status	Reason	Required evidence
Tooling and desktop-profile CI	FAIL	current jobs terminate in executed test steps with exit code 1	successful repetition on the corrected commit
Windows package	FAIL	frontend installation failed in the native Windows job	successful native build and artifact upload
Release Validation	not verified	no manual or tag run is available	successful non-publishing run on a fixed commit
Compose startup	not verified	only the model and master images were checked in CI	`docker compose up`, health/readiness evidence, and controlled teardown
Branch Protection	not verified	the public API requires authentication	authorized evidence of the required checks for `main`
Signing, notarization, updater	outside the scope	product-specific identity and protected credentials are intentionally absent	protected native release pipeline for each target platform
Production operation and auth/RBAC	outside the scope	cloud environment, business model, and security model are product-specific	product-specific acceptance including credentials and deployment

Source: Author's own compilation based on kleiveist (2026i, 2026k) and the current GitHub Actions runs.

In simple terms: What remains to be checked externally

External checks require services, permissions, or infrastructure outside the local template. *Branch Protection* consists of rules that, for example, allow a code branch to be changed only after checks have passed. Deployment is the controlled process of making an application available in its target environment. Signing adds a verifiable digital signature to a package; during notarization, an external platform authority additionally checks the package. An updater distributes new versions to applications that are already installed. Production operation comprises continuous, monitored use by real users. For the baseline examined, these points have not been fully demonstrated: Branch Protection could not be read, the generated packages were unsigned, and no production deployment was performed.

7 Evaluation of the Technology Template

7.1 Productivity Effects

The evaluation compares the requirements, implemented mechanisms, and evidence. The criteria are reuse, standardization, variability, maintainability, testability, reproducibility, developer experience, extensibility, and platform coverage. Complexity, operational maturity, and documentability supplement this assessment. The project's internal acceptance report provides project-specific evidence, but no independent replication (kleiveist, 2026k). This distinction keeps observed mechanisms, executed verification activities, and inferred effects separate. Automation alone is not considered a measurable productivity gain.

Here, productivity encompasses the effort required to obtain a runnable initial project, recurring setup decisions, the development flow, and subsequent maintenance. A single activity metric does not represent it adequately (Forsgren et al., 2021). Conceptually, the effect can therefore be expressed as

$$\textit{Productivity benefit} = \textit{avoided project effort} - \textit{template maintenance effort} - \textit{additional complexity}$$

In the absence of time and effort measurements, it cannot be quantified. The relationship also shows that centralized maintenance and additional complexity can reduce the project effort avoided.

At project initialization, `control.py` combines profile selection, generation, system checks, and installation. The acceptance results provide evidence for these steps across all five profiles and for the rejection of incompatible PostgreSQL selections (kleiveist, 2026k). The mixed CI results for the later HEAD limit the transferability of this evidence. Less manual setup is therefore plausible, but a specific amount of time saved has not been demonstrated.

During development, profile-dependent entry points standardize startup, testing, and builds. The shared frontend, configuration contract, and clear responsibilities encourage reuse; specialized tools and platform prerequisites remain necessary. Central changes benefit future projects, but require shared maintenance of profiles, generator, tests, and documentation. Projects that have already been generated are not updated automatically. The benefit is thus distributed across project initialization, ongoing development, and subsequent baseline maintenance. It is particularly plausible when several technically related projects use the same foundations.

Quantitative evidence is lacking. A future controlled evaluation would need to compare similar projects with and without the template in terms of initialization and onboarding time, setup errors, failed builds, proportion of reuse, and maintenance time. It should also record the time to the first domain-specific feature and the number of manual setup decisions. Only such comparative data would permit a statement about the magnitude of the effect.

In simple terms: Productivity and measurement

Developer productivity describes how effectively a development team turns its time into useful results.
A template can simplify recurring setup and thereby create potential for productivity gains.
Such potential is initially only a possible benefit, not a measured time saving.
A quantitative measurement would require comparable figures, for example on setup time, errors, and maintenance effort.
Because such comparative data are unavailable, the case study assesses the effect as plausible, but not as demonstrated numerically.

7.2 Strengths

The shared baseline combines centralized maintenance with clear boundaries between the frontend, backend, desktop, persistence, and tooling layers. The shared frontend addresses A-02 and A-05 without burdening every profile with all components. This separation supports independent tests and product-specific extensions without duplicating the shared presentation layer.

Five profiles represent recurring platform configurations. Dependencies prevent the use of PostgreSQL without a backend. The acceptance results for all base profiles and for the permitted and prohibited PostgreSQL combinations support this model for the limited current catalog, but not for an arbitrary number of future capabilities (kleiveist, 2026k).

`control.py` standardizes the entry point while keeping the specialized tools visible. The manifest, configuration contract, tests, and build paths increase traceability. The byte-identical generation of two `desktop-cloud + postgres` projects provides concrete evidence of this, albeit limited to one combination and environment (kleiveist, 2026k). Profile-dependent skipping of absent layers keeps

the shared command interface aligned with the actual project scope.

Table 7 contrasts the strengths with the limits of their applicability.

Table 7: Evidence-based comparison of strengths and limitations

Area	Strength and evidence	Limitation and evidence	Significance
Architecture	Separate frontend, backend, Tauri, and persistence boundaries; a shared frontend.	Shared schemas are not consumed automatically by the frontend and backend.	Reuse with clear responsibilities; contract changes remain manual.
Variability	Five declarative profiles; dependencies and three permitted PostgreSQL combinations have passed acceptance.	Only one selectable capability; the direct CLI does not strictly enforce the distinction between `optional` and `selectable`.	Controlled selection in the current catalog, with limited implications for extensions.
Tooling	Shared profile-dependent entry point for diagnostics, installation, startup, testing, and building.	Several specialist tools and native prerequisites remain necessary.	Fewer distinct interaction paths, but no technology-independent workflow.
Quality	Tests, the profile matrix, build paths, and the release gate are recorded in the acceptance report.	Current tooling, desktop-profile, and Windows jobs fail; Branch Protection has not been demonstrated.	A sound local verification basis, but no consistently successful remote evidence.
Reproducibility	Two identically configured projects were byte-for-byte identical in the documented experiment.	Evidence covers only one combination and one examination environment.	Specific evidence without claiming complete cross-platform reproducibility.
Operation and platforms	Web and image builds, PostgreSQL integration, and unsigned Linux/macOS packages are documented.	Compose startup, production cloud operation, a Windows package, signing, and notarization remain outstanding.	Template maturity must not be equated with production readiness.
Maintenance	Core, profiles, tooling, tests, and documentation reside in one baseline.	Changes may affect multiple profiles; product projects receive no automatic updates.	Central maintenance is consolidated, but requires broad regression testing and transfer processes.

Source: Author's own evaluation based on Chapters 3–6, the acceptance report, and the current CI runs (GitHub, 2026a, 2026c, 2026d, 2026e; kleiveist, 2026k).

7.3 Weaknesses and Limitations

Technical weaknesses, deliberate scope boundaries, and unverified areas must be distinguished. Only the first category directly denotes a disadvantage of the implementation. Although scope boundaries create a need for adaptation, they do not constitute an unmet requirement. A lack of verification does not yet permit either a positive or a negative technical judgment.

Consolidation in the master repository increases internal complexity: changes to the core, generator, or catalog require regression tests across several profiles. In addition, the frontend and backend do not use the shared JSON schemas at runtime; contract changes remain manual. Direct CLI selection distinguishes `optional` and `selectable` less clearly than the dialog does. Both discrepancies create risks of maintenance errors or incorrect operation. The examined HEAD also has conflicting evidence: the web profiles and the local tooling run passed, while the external tooling, desktop-profile, and Windows checks failed. This finding does not retroactively call the earlier, commit-specific acceptance into question, but it prevents a fully successful assessment of the more recent state.

The deliberate exclusions are a backend in `web-only`, a cloud API in `desktop-local`, and PostgreSQL without a backend. Domain logic, authentication, role models, mandatory persistence, cloud providers, and native domain-specific commands likewise remain product-specific. The new decision framework for product persistence provides no universal local-file, embedded-database, or synchronization implementation. This deliberate scope boundary is an extension path, not an implementation defect (kleiveist, 2026h).

Compose startup, production deployment, Release Validation, and Branch Protection have not been conclusively verified. Unsigned Linux and macOS packages were built, while Windows packaging failed. Signing, notarization, publication, and production cloud operation remain product- or infrastructure-specific (GitHub, 2026a, 2026c; kleiveist, 2026k). The packaging results therefore demonstrate neither signed releases nor production cloud operation. The subject of the evaluation is the template baseline, not the production readiness of a product. The remaining risks therefore primarily concern operations, platforms, and release processes.

In simple terms: Weaknesses, scope boundaries, and evidence

A technical weakness is an actual disadvantage of the existing implementation.
A scope boundary, by contrast, deliberately defines what the template is not intended to cover.
A lack of verification means that a function has not yet been tested or evidenced adequately; it is therefore not automatically defective.
A regression is an error introduced by a change in an area that previously worked.

7.4 Maintenance Effort

The single-repository strategy consolidates the core, features, profiles, generator, tests, and documentation. This avoids parallel baseline changes and benefits future projects. Repositories that have already been generated must, however, incorporate fixes independently. The source of truth thus reduces the number of starting points that require maintenance, but not the effort needed to migrate existing products.

Central maintenance encompasses several languages, dependency managers, toolchains, platforms, and release cycles. This diversity follows from the web, backend, desktop, and deployment objectives, but increases the expertise required, the breadth of testing, and dependency on the environment. Native builds and external services are more maintenance-intensive than a purely web-based, technologically homogeneous starter.

Five profiles, six features, and one selectable capability currently limit the combination space; acceptance covers five base paths and three permitted PostgreSQL extensions (kleiveist, 2026k). Additional capabilities expand both the dependencies and the test matrix. A combinatorial explosion has not been demonstrated, but increasing maintenance effort is plausible. With such extensions, the catalog, resolver, configuration contract, test matrix, and documentation must remain mutually consistent.

Central maintenance is therefore more demanding than maintenance of a small, homogeneous starter, but can consolidate repeated maintenance of the baseline. Its economic viability depends on frequency of use, similarity between projects, and maintenance time; cost data are unavailable.

In simple terms: Central maintenance

A source of truth is the single authoritative source from which the shared technical starting point is derived.
Maintenance effort comprises all work needed to keep this foundation current, consistent, and functional.
A regression test checks whether a change has unintentionally damaged parts that already worked.
A toolchain is the complete set of development, testing, and build tools required.
In the examined template, the central source avoids parallel maintenance of the baseline, but multiple toolchains and projects that have already been generated continue to create maintenance work.

7.5 Comparison with Alternative Template Strategies

The architectural strategies are compared in terms of reuse, consistency, variability, entry complexity, maintenance, testability, and adoption of updates (Clements & Northrop, 2001; Krueger, 1992). The qualitative levels do not form an overall ranking; high entry complexity, for example, is a disadvantage.

Separate template repositories remain individually manageable, but can diverge. A *copy-and-paste starter* minimizes template logic, but provides no channel through which subsequent improvements can be propagated. GitHub templates accordingly create a separate, unrelated history (GitHub, 2026b). Separate repositories therefore give greater weight to organizational isolation than to the central consistency of shared components.

Framework-specific starters such as `create-vite` offer a simple entry point into a narrow area. The backend, desktop layer, persistence, and overall workflow must be added separately when needed (VoidZero Inc. and Vite Contributors, 2026). For a small, homogeneous frontend, this lower entry complexity may be more appropriate than the full-stack approach under examination.

A *monolithic universal template* simplifies selection, but burdens small projects with unused components. The examined *single repository with profiles and capabilities* combines centralized maintenance with selected paths. This increases the complexity of the master repository, resolver, and test matrix; updates do not reach generated projects automatically. It thus combines centralized variability before generation with the autonomy of a copied starter afterward.

Table 8: Qualitative comparison of template strategies

Strategy	ongoing reuse	Consistency	Variability	Entry complexity	Maintenance consolidation	Updates to existing projects
Separate template repositories	medium	medium	medium	low	low	manual for each template or project
Copy-and-paste starter	low after copying	low	high	low	low	no automatic adoption
Framework-specific starter	high within a narrow scope	high within a narrow scope	low to medium	low	high within the starter	usually manual after generation
Monolithic universal template	medium	high	low	high	high	depends on the selected derivation method
Single repository with profiles	high within the defined scope	high	high, controlled	medium	high	centralized in the master, manual in derived projects

Source: Author's own qualitative comparison based on Krueger (1992), Clements and Northrop (2001), GitHub (2026b), and VoidZero Inc. and Vite Contributors (2026).

The profile-based approach is particularly suitable for recurring, technically related projects. Small individual projects may benefit from framework starters, while organizationally separate product lines may benefit from separate repositories. There is no universal winner, but rather a set of different trade-offs.

In simple terms: Template strategies

A trade-off is a choice in which an advantage is accompanied by a disadvantage.
A framework starter provides a simple entry point for a narrowly defined technical area, such as the frontend alone.
A monolithic template contains all intended components together, even if a small project does not need some of them.
With a copy-and-paste starter, the initial foundation is copied once; subsequent improvements to the template do not reach the project automatically.
The profile strategy in this case study selects components before generation, but accepts additional maintenance and selection complexity in return.

7.6 Overall Evaluation

Table 9 distinguishes criteria that are largely or partially addressed from those that have not been conclusively demonstrated. These levels are qualitative and measure neither quality nor productivity numerically.

Table 9: Consolidated evaluation of the technology template

Criterion	Evidence base	Assessment	Key limitation
Reuse and standardization	Shared Core, five profiles, generator, and uniform command interface	largely addressed	Applies to the defined stack; derived projects are not updated automatically.
Variability and extensibility	Profile presets, capability dependencies, and verified PostgreSQL combinations	largely addressed	Small current catalog; extensions increase the maintenance and testing scope.
Testability and reproducibility	Documented profile, integration, and build acceptance, plus a byte-for-byte identical generation comparison	partially addressed	Some current remote jobs failed; reproducibility experiment covers only one combination.
Developer experience and documentability	`control.py`, diagnostic paths, versioned manifests, and structured documentation	partially addressed	Onboarding and interaction effort were not measured empirically; multiple toolchains remain visible.
Maintainability	One central baseline with shared tests and documentation rules	partially addressed	Technology diversity, variant regression, and manual interface synchronization.
Platform coverage and operational maturity	Evidence covering web, backend, PostgreSQL, container images, Linux, and macOS; provider-neutral boundary	not conclusively demonstrated	Windows package, Compose startup, signing, notarization, and production deployment have not been evidenced.

Source: Author's own evaluation based on the preceding chapters, kleiveist (2026k), and the CI runs evaluated in Section 6.2.

The baseline is technically viable for the defined profiles. The core, presets, dependencies, and tooling support reuse, standardization, and flexibility. The acceptance results support generation, installation, tests, builds, PostgreSQL combinations, and Linux packaging (kleiveist, 2026k). More recent external runs add evidence for web, container, Linux, and macOS paths, but include failures in tooling, desktop-profile, and Windows jobs. The evidence is therefore robust, but varies in scope by commit and platform.

Limitations include the complexity of central maintenance, manually synchronized contracts, and the absence of updates for generated projects. Windows packaging, signed releases, notarization, and production cloud operation have not been demonstrated. Deliberately omitted provider and domain logic, by contrast, is not a defect. Table 9 makes the distinction between an addressed requirement and the outstanding scope of verification explicit.

Automation and shared workflows provide plausible, but unquantified, potential for productivity gains. The current state is suitable as a standardized foundation for the intended project families, but not as a finished production application or a universal template. Whether the central investment is economically worthwhile remains unresolved in the absence of usage and cost data.

In simple terms: Maturity and platform coverage

Operational maturity means that an application can be operated and monitored reliably under the intended conditions.
Production readiness goes further: security, approvals, and the specific requirements of a finished product must also be met and evidenced.
Platform coverage describes the access channels and operating systems for which a solution is intended and has been tested.
The case study demonstrates this coverage to different extents depending on the commit and platform.
The template is therefore a standardized technical foundation, but not a finished production application.

8 Application Scenarios and Transfer to Customer Projects

8.1 Suitable Project Types

Suitability depends on access channels, the backend and persistence, deployment, native integration, and infrastructure, security, and operational needs. The evidence covers combinations that can be generated and verified profile paths. By contrast, the levels *well suited*, *conditionally suited*, and *not primarily intended* qualitatively assess the distance from the baseline, not the probability that a project will succeed (kleiveist, 2026g, 2026k). The more central requirements lie outside the available components and evidence, the greater the adaptation effort and remaining risk.

`web-only` is suitable for browser-based frontends without a template backend. `web-cloud` adds a central API and provider-neutral deployment structure, for example for internal business and data applications. PostgreSQL remains optional; authentication, authorization, domain models, and production infrastructure are product-specific. The profile therefore covers the initial technical structure, not the particular business application.

`desktop-local` is appropriate for local tools without a central backend, provided that native functions can be added within a manageable scope; a local database is absent. `desktop-cloud` adds central services, and `full-platform` also adds the browser channel. The primary application domain therefore comprises web-, cloud-, and desktop-oriented business and full-stack applications. For all desktop variants, more extensive native functions must be added and verified specifically.

Category	Project types	Decisive criterion
WELL SUITED	Browser frontend; web + backend/cloud; local desktop tool; desktop + cloud; web + desktop + cloud	Access channels and runtime layers match a verified profile; domain-specific and operational functions still need to be added.
CONDITIONALLY SUITED	high proportion of native code; highly customized infrastructure; stringent compliance or security requirements	A profile covers only part of the requirements; substantial extensions or additional evidence are required.
NOT PRIMARILY INTENDED	hard real-time systems; game-engine-based games; native mobile apps; core firmware and embedded systems	The underlying runtime or platform architecture falls outside the Vite/FastAPI/Tauri baseline.

Figure 17: Qualitative suitability overview by architectural fit and adaptation effort

Source: Author's own transfer assessment based on the profile definitions and technical acceptance (kleiveist, 2026g, 2026k).

Figure 17 and Table 10 summarize the classification. *Suitable* means only that the baseline covers essential platform components. Domain logic, user interface, integrations, product security, and operations remain open, as does the economic assessment.

Table 10: Mapping of technical project types to the template baseline

Project type	Suitability	Profile	Required additions and limitations
Browser frontend	well suited	`web-only`	Domain logic and UI; external APIs are possible, but the baseline provides no FastAPI backend.
Web application with API/cloud	well suited	`web-cloud`	Authentication, domain models, and production operation; PostgreSQL only if relational persistence is required.
Local desktop tool	well suited	`desktop-local`	Native commands, packaging, and signing are product-specific; no local database is included.
Desktop application with central backend	well suited	`desktop-cloud`	Production API, security, and signing; PostgreSQL remains optional.
Browser + desktop + cloud	well suited	`full-platform`	Channel-specific UX and tests; the profile sets no persistence provider, while PostgreSQL is optional.
Application with a high proportion of native code	conditionally suited	desktop profile or another foundation	Substantial effort for platform APIs, permissions, system-level services, or a native UI.
Highly specialized infrastructure	conditionally suited	depends on the partial architecture	Suitable only if the client or an individual service can be meaningfully isolated; infrastructure is not prescribed.
Hard real-time systems, game-engine-based games, native mobile applications, or core embedded systems	not primarily intended	no profile	The underlying runtime and platform requirements fall outside the baseline.

Source: Author's own qualitative mapping based on kleiveist (2026g, 2026h, 2026k).

In simple terms: The technical foundation of a business application

A business application supports workflows in an organization, for example by allowing data to be processed and shared.
Domain logic comprises the rules and processes of the particular area of business.
An initial technical foundation provides general components such as a user interface, backend, or deployment structure on which a product can be built.
The template provides such a foundation, but neither the domain logic nor the finished business application.

8.2 Unsuitable and Conditionally Suitable Project Types

Projects requiring extensive operating-system integration, a platform-specific UI, system-level services, or drivers are only conditionally suited. The Tauri shell does not provide sufficient evidence of an adequate native foundation for such projects. Highly distributed infrastructure and specialized event-streaming or HPC requirements may likewise require a different architecture; at most, the baseline remains usable for a clearly delimited part.

Hard real-time systems, graphically intensive games, firmware, embedded systems, and native mobile applications do not belong to the primary target domain. Ordinary live updates do not turn a business application into a real-time system. Individual mobile or hardware-oriented functions are not excluded, but the template does not provide their technical foundation. The decisive question is therefore not whether a single characteristic is present, but whether the baseline supports the technical core of the product.

High security or compliance requirements do not preclude use of the baseline, but require additional evidence concerning protection requirements, audits, certification, and legal compliance. Compose validation and master images are evidenced, but Compose startup and production deployment are not (GitHub, 2026a). Linux and macOS packages were built without signing; the Windows packaging path failed (GitHub, 2026c). Signing and notarization remain product-specific (kleiveist, 2026i). The required TypeScript, Python, Rust, database, and deployment expertise must also be assessed. If this evidence or expertise is unavailable, the project is at most conditionally suited.

In simple terms: Specialized project types

Native integration means directly using specific functions of an operating system or device.
In a hard real-time system, a response must meet a fixed time limit under all intended conditions.
A game engine provides specialized tools for graphically intensive games.
An embedded system is a computer within a device; its hardware-oriented software is called firmware.
A native mobile app is developed specifically for a mobile operating system, whereas HPC refers to particularly compute-intensive tasks performed on high-performance systems.
Compliance means meeting binding legal, contractual, or organizational requirements and, when necessary, providing evidence that they have been met.
For projects with such a technical core, the template does not provide a complete foundation; projects with high compliance requirements need additional checks.

8.3 Analysis of Customer Requirements

The analysis begins with the functional objective, user groups, workflows, and deployment environment. Only then are the requirements classified in technical terms. Shared data processing, for example, may require a backend and persistence, but does not yet determine the use of PostgreSQL.

Browser and desktop access, server functions, persistence, offline needs, native functions, external APIs, operations, deployment, security, and compliance are recorded separately. Despite lacking a cloud backend, `desktop-local` includes neither a local persistence provider nor a local data model or synchronization function. Table 11 consolidates the guiding questions.

Table 11: Compact checklist for technology-neutral requirements analysis

Area	Guiding question	Influence on selection
Domain functionality and use	Which workflows, user groups, and locations of use must be supported?	Determines the product scope; does not yet determine a profile.
Access channels	Will the product be used in a browser, as a desktop application, or through both channels?	Distinguishes web, desktop, and full-platform paths.
Central services	Must functions or shared data be available on the server side?	Requires a backend-capable cloud profile.
Persistence and offline operation	Which user data is created; which is structured and which consists of files or assets; what is the source of truth; are offline operation, relational queries, or synchronization required?	Determines the provider, schema, migration, backup, and recovery independently of the profile; PostgreSQL requires a backend.
Native integration	Which operating-system APIs, devices, or platform-specific interfaces are required?	Determines the need for Tauri and the distance from the native baseline.
Integration and operation	Which external systems, target environments, and scaling and operational requirements exist?	Produces product-specific adapters and infrastructure decisions.
Security and compliance	Which protection, evidence, audit, or regulatory requirements apply?	Requires additional measures and evidence independently of the profile.

Source: Author's own compilation.

The result comprises platform components and gaps, for example in authentication, data models, integrations, observability, backups, infrastructure, or signing. Only then can a decision be made between a suitable profile, a limited extension, and a solution outside the baseline. The method therefore deliberately follows the sequence of requirements, technical classification, and only then profile or technology selection.

In simple terms: Customer requirements

A customer requirement describes what a product must do or the conditions under which it must operate.
A functional requirement specifies a desired function; a technical requirement defines, for example, the platform, security, or operations.
Offline operation means that required tasks should remain possible without a network connection.
An external API is an interface to another system.
Monitoring and observability help teams observe the state and behavior of a running application and understand errors.
A backup is a safeguarded copy from which data can be restored.
Compliance requirements record binding rules; an audit is a traceable examination of whether they have been followed.
These requirements are collected before profile selection because the template does not already cover all of them.

8.4 Selecting the Project Profile

The access channels determine the profile: `web-only` denotes a browser without a backend, `web-cloud` a browser with a backend, `desktop-local` a desktop application without a cloud backend, `desktop-cloud` a desktop application with a backend, and `full-platform` the browser and desktop channels together. There is no profile for a backend without provider-neutral deployment files.

The smallest suitable profile is selected. PostgreSQL is then assessed as a separate dimension and added to a backend profile only when needed. This is not permitted for `web-only` and `desktop-local` (kleiveist, 2026g).

The documentation records whether the profile is a complete match, requires specified extensions, or fails to meet central requirements. In the last case, a different architecture or clearly delimited partial use should be examined instead of selecting the largest profile. Team expertise and economic constraints supplement this technical decision, but do not change the meaning of the profiles.

In simple terms: The smallest suitable profile

The smallest suitable profile contains only the technical components that a project actually needs.
The technical selection is based first on whether a browser, desktop application, and backend are required.
PostgreSQL is then assessed separately and added only to a suitable backend profile.
This approach avoids unnecessary components and the additional complexity they create.
If a profile does not meet central requirements, a different architecture or clearly limited partial use should be examined instead of selecting the largest profile.

8.5 Generating a Customer Project

`init` creates a separate project from the profile, capability, and project identity. Its active manifest is named `project-profile.toml`. Generation is the handover point to product development, not the completion of the product (kleiveist, 2026g, 2026m). For desktop projects, these inputs include a valid application ID.

In the target directory, `doctor`, `install`, `run`, and `test --suite all` check the baseline. Profile-dependent Tauri, database, or container checks supplement the transfer shown in Figure 18.

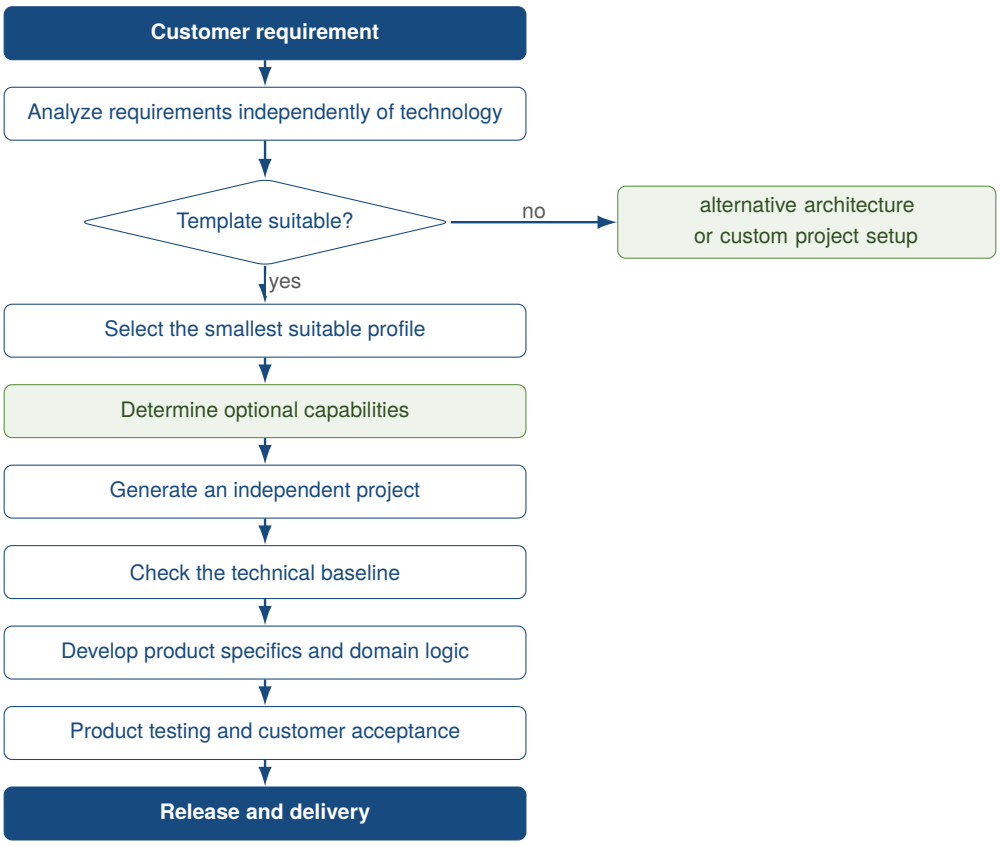

Figure 18: Vertical transfer process from customer requirement to product project

Source: Author's own illustration.

In simple terms: From the template to a product project

Generation creates an independent project from the selected template configuration.
This step is the transition from the shared technical template to development of the specific customer product.
The generated project contains a verifiable starting point, but is not yet a finished product.
Domain-specific features, product-specific requirements, and the product's own evidence are added to the new project only afterward.

8.6 Transition to Product Development

After generation, the independent product project decides on domain models, user data, persistence providers, assets, schemas and migrations, backup and recovery, the security model, and, where applicable, synchronization (kleiveist, 2026h). Business rules, the user interface, integrations, operations, scaling, monitoring, data protection, and signing also remain product tasks. The baseline structures this work, but does not solve these product-specific tasks.

The master repository and the product have separate lifecycles. There is no permanent update or synchronization mechanism. Changes to the master repository must be transferred to products deliberately; product changes should flow back into the baseline only after a generalization and verification review.

Template acceptance and product acceptance remain separate. The commit-specific acceptance provides evidence for generation, installation, tests, and selected baseline builds, but not for deployment,

signing, or functional and security requirements (kleiveist, 2026k). The product requires its own tests and evidence for these areas. They must cover the specific customer requirements, integrations, and operating conditions.

In simple terms: Product development responsibilities

In the product project, the domain logic explained above is implemented for the specific customer.
Scaling ensures that the application can continue to provide sufficient performance as its use grows.
Monitoring and backups must be configured for the specific operating conditions. Monitoring makes problems visible. Backups make it possible to restore lost or damaged data.
Data protection governs the lawful and secure handling of personal data.
Product acceptance verifies whether the finished product meets the specific customer requirements and operating conditions.
Acceptance of the template baseline does not replace this separate product verification.

9 Conclusion

9.1 Summary of Results

The master template addresses repeated setup effort and diverging project bases through a shared core and declarative variability. It consolidates generation, configuration, boundaries of responsibility, workflows, and maintenance.

Five profiles combine the shared Vite frontend with a profile-dependent FastAPI backend and an optional Tauri layer; PostgreSQL can be selected only with a backend. `control.py` standardizes the development path. Generated projects remain independent and receive no automatic updates from the master repository.

The local acceptance of an earlier commit supports the profile, PostgreSQL, test, web build, and Linux packaging paths. The current external CI is only partially successful; further evidence is either unavailable or product-specific. The evidence therefore demonstrates a baseline with controlled variability, not production readiness or empirical suitability for customer projects.

9.2 Answers to the Research Questions

Research Question 1: Requirements Addressed. The template addresses the functional and technical, quality, and operational requirements through a baseline that can be generated, clearly defined components, profiles, shared workflows, centralized maintenance, and versioned configuration. Separate runtime and verification paths support this; domain logic, authentication, authorization, and production operation remain product-specific.

Research Question 2: Variants for Web, Cloud, and Desktop. Five presets combine the shared frontend with backend, cloud, and Tauri components; PostgreSQL is added only when backend capability is present. Generated projects predominantly contain the paths they require. `desktop-cloud` and `full-platform` are scaffolded identically at the technical level and differentiated semantically.

Research Question 3: Reuse and Standardization. Centralized maintenance of implementations, profiles, the generator, configuration, tests, and documentation promotes reuse and consistent initial states across projects. The benefit applies to the baseline and future generations; existing product projects receive no automatic updates.

Research Question 4: Reduction of Recurring Effort. Automated or standardized profile selection, generation, diagnostics, installation, startup, testing, and builds provide plausible potential for reducing recurring effort. Time, effort, and comparative data are lacking, however; a productivity improvement has not been demonstrated when central maintenance and multiple toolchains are taken into account.

Research Question 5: Suitability for Project Types and Customer Requirements. The template is suitable as a foundation for related web, desktop, and combined business applications whose platform and cloud needs can be represented. Persistence needs, persistence providers, and the source of truth are determined separately for the specific product. Highly native applications, games, and specialized real-time systems lie outside the primary target domain. The qualitative classification is not a universal assurance of suitability.

Research Question 6: Limitations, Maintenance Effort, Risks, and External Dependencies. Limitations include the growing breadth of regression testing, manual contract synchronization, insufficiently strict separation between optional and selectable capabilities, the absence of product updates, and the effort of maintaining multiple toolchains. The tooling, desktop-profile, and Windows CI paths are currently failing; Release Validation, Compose startup, and Branch Protection are still missing. Signing, notarization, the updater, production deployment, and the security model remain product tasks.

9.3 Critical Reflection

The project-specific case study has limited generalizability. The documentation, acceptance, and local verification originate from the project context; the developer perspective may influence the selection of evidence. Literature and official documentation are no substitute for independent replication. Commit-specific findings remain valid, but their transfer to other repository states or projects is limited.

The evidence is heterogeneous: an earlier acceptance performed predominantly on Linux contrasts with mixed current CI results. Without test-coverage measurement, Compose startup, and successful platform runs, there is no comprehensive evidence of quality or production readiness; unsigned packages are no substitute for product releases.

Three principles in particular are transferable: a centralized baseline, declarative variability, and shared tooling entry points. Productivity and suitability were assessed only qualitatively; time measurements, comparison groups, customer cases, and economic data are lacking. These gaps particularly limit statements about the actual effects in everyday project work.

9.4 Outlook

In the short term, the failing tooling, desktop-profile, and Windows CI paths should be corrected on a fixed commit. Compose startup with health and readiness checks, Release Validation, and Branch Protection should supplement the technical evidence without claiming production readiness.

In the medium term, specific products require signing, notarization, and an updater. Credentials, deployment, authentication, and RBAC also remain product-specific. Three measures are appropriate for the baseline: automate the use of shared contracts, make capability selection stricter, and synchronize generated projects in a controlled manner.

In the long term, a comparative study across projects should measure onboarding, time to the first build, setup errors, and maintenance effort. Real-world customer projects could test the profile and suitability criteria.

9.5 Conclusion

The template is a centrally maintained, profile-based baseline for the defined web, cloud, and desktop variants. The shared core, capabilities, and tooling standardize recurring project workflows.

Controlled variability provides plausible potential for productivity gains. Without empirical data, its magnitude and suitability for customer projects can be substantiated analytically only for the delimited target domain.

The evidence extends only to the examined technical baseline. The local acceptance, current CI failures, outstanding external verification activities, and product-specific release and security tasks do not permit it to be equated with a finished, cross-platform-verified product.

References

Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Retrieved August 8, 2026, from https://alembic.sqlalchemy.org/en/latest/tutorial.html

Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Retrieved August 8, 2026, from https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/

Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The space of developer productivity: There's more to it than you think. *Queue*, *19*(1), 20–48. https://doi.org/10.1145/3454122.3454124

GitHub. (2026a, August 7). *Core ci, run 31168215013* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168215013

GitHub. (2026b). *Creating a repository from a template* [GitHub Documentation]. Retrieved August 9, 2026, from https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template

GitHub. (2026c, August 7). *Desktop ci, run 31168214763* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168214763

GitHub. (2026d, August 7). *Postgresql integration, run 31168216208* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168216208

GitHub. (2026e, August 7). *Profile matrix, run 31168215737* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168215737

ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard ISO/IEC 25010:2023). International Organization for Standardization. Retrieved August 8, 2026, from https://www.iso.org/standard/78176.html

Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Retrieved August 8, 2026, from https://doc.rust-lang.org/book/

kleiveist. (2026a, August 7). *Central project control cli* [CLI implementation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/tools/control.py

kleiveist. (2026b, August 7). *Container builds and local production simulation* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/container-builds.md

kleiveist. (2026c, August 7). *Continuous integration* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/ci.md

kleiveist. (2026d, August 6). *Database feature* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/database-feature.md

kleiveist. (2026e, August 6). *Deployment architecture* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/deployment-architecture.md

kleiveist. (2026f, August 6). *Framework architecture* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md

kleiveist. (2026g, August 6). *Project profiles* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md

kleiveist. (2026h, August 13). *Provider-neutral persistence architecture* [Project documentation in the Template-Projekte repository, commit 7257776]. Retrieved August 13, 2026, from https://github.com/kleiveist/Template-Projekte/blob/7257776c21606f187156e1451ebe9fa3d851600c/docs/def/persistence-architecture.md

kleiveist. (2026i, August 7). *Release and desktop packaging model* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/release-model.md

kleiveist. (2026j, August 6). *Runtime configuration* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/configuration.md

kleiveist. (2026k, August 7). *Template final acceptance* [Technical acceptance report in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md

kleiveist. (2026l, August 7). *Template-projekte: Full-stack project template* [GitHub repository, examined commit a4487ac]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte

kleiveist. (2026m, August 6). *Tooling guide* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md

Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, *24*(2), 131–183. https://doi.org/10.1145/130844.130856

Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, *39*(2), 62–70. https://doi.org/10.1109/MS.2021.3073045

Microsoft. (2026, July 27). *The typescript handbook* [TypeScript Documentation]. Retrieved August 8, 2026, from https://www.typescriptlang.org/docs/handbook/intro.html

PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Retrieved August 8, 2026, from https://www.postgresql.org/docs/current/index.html

Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Retrieved August 8, 2026, from https://fastapi.tiangolo.com/features/

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, *14*(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

SQLAlchemy Authors and Contributors. (2026, June 15). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Retrieved August 8, 2026, from https://docs.sqlalchemy.org/en/20/core/engines.html

Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Retrieved August 8, 2026, from https://v2.tauri.app/security/capabilities/

Tauri Contributors. (2026b, July 22). *What is tauri?* [Tauri 2 Documentation]. Retrieved August 8, 2026, from https://v2.tauri.app/start/

VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Retrieved August 8, 2026, from https://vite.dev/guide/